National Sun Yat-sen University
Department of Electrical Engineering

114-1 Practical Digital System Design

HW1 Combinational 14-bit adder/subtractor

Name:EnPing Chung
Student ID:B133015012

Table of Contents

I. Design Method

i. ripple/carry-lookahead

ii. unsigned/signed with overflow

II. Circuit Architecture

III. Design Comparison

i. gate count

ii. longest critical path

IV. Testbench Design

V. Waveform Analysis

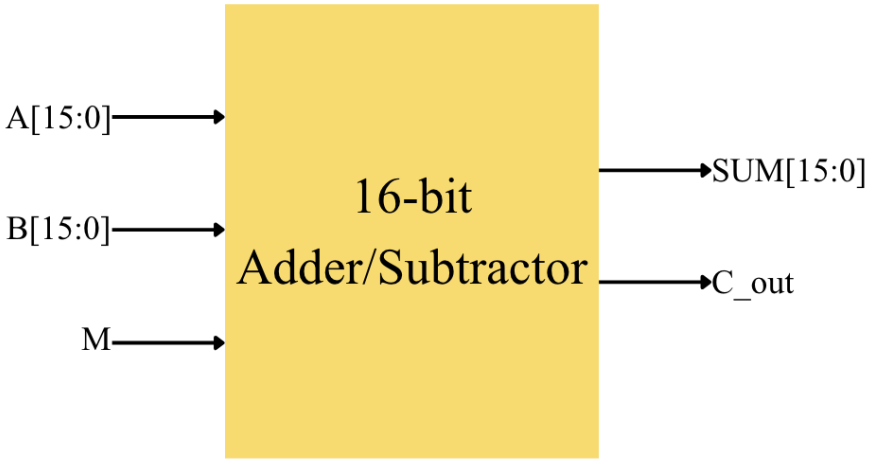
VI. Experience and Reflection

I. Design Method

The goal of this project is to design and verify a 16-bit combinational adder/subtractor using gate-level Verilog. The circuit accepts two 16-bit inputs, A and B, and a control signal, M, where M = 0 selects addition and M = 1 selects subtraction. Each design is constructed from four 4-bit adder blocks and produces a 16-bit result and a carry-out signal.

Both ripple-carry and carry-lookahead architectures are implemented, with unsigned versions and signed versions that include overflow detection. These designs are compared in terms of gate count and critical-path delay to evaluate the tradeoff between hardware complexity and speed. Testbench results and simulation waveforms are used to verify arithmetic correctness, overflow detection, and propagation delay.

Below is a black box diagram of the intended design:



Phase I: Design of Full-Adder

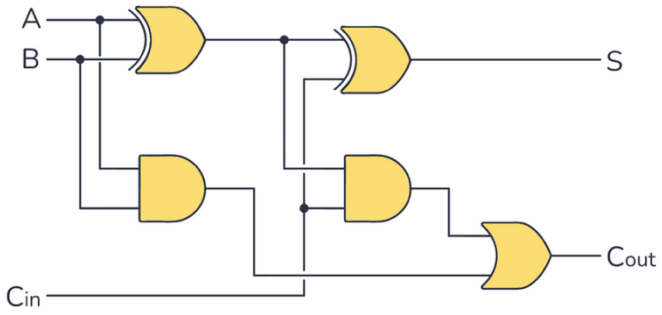
The design begins with a 1-bit full adder, which serves as the basic building block for multi-bit addition. It takes three inputs: two operand bits, A and B, and a carry-in, Cin. It produces a sum bit, S, and a carry-out, Cout, according to the following equations:

$S = A \oplus B \oplus Cin$
$Cout = (A \cdot B) + (Cin \cdot (A \oplus B))$

Here, ⊕ represents XOR, · represents AND, and + represents OR. The circuit first calculates A ⊕ B, then combines this intermediate result with Cin to produce the sum. A carry-out occurs when both operand bits are 1, or when one operand bit is 1 and an incoming carry is present. This implementation uses two XOR gates, two AND gates, and one OR gate.

Four full adders are connected to form a 4-bit ripple-carry adder. Each stage passes its carry-out to the carry-in of the next stage, starting from the least significant bit. Four of these 4-bit blocks are then connected to construct the 16-bit ripple-carry adder used in the complete adder/subtractor.

Below is a diagram and truth table of a Full-Adder:



=full_adder truth table=

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit Analysis of Full Adder

Gate Count

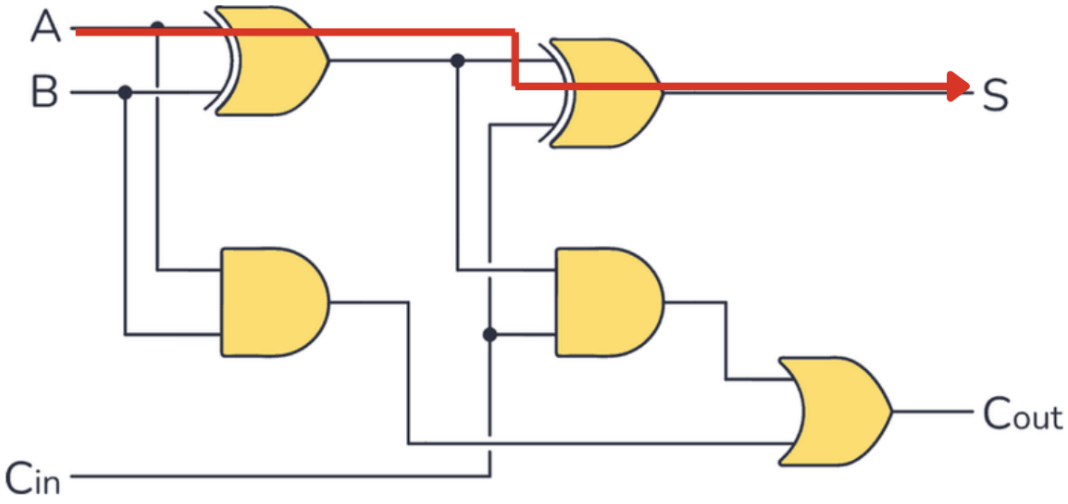
From this implementation, the gate usage is:
XOR gates: 2
AND gates: 2
OR gates: 1
NOT gates: 0

Therefore, the total number of gates in one full adder of my design is 5.

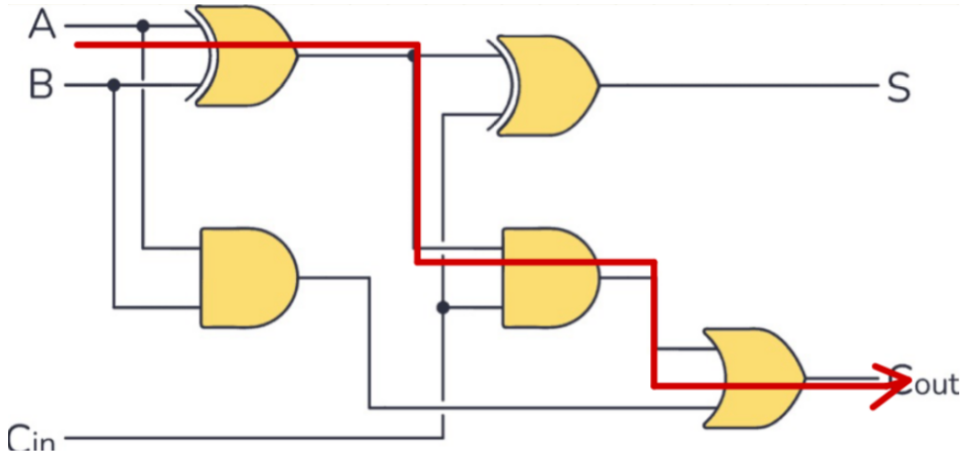
Longest Critical Path

For timing analysis, assuming each gate has one unit delay, the output S is produced through two cascaded XOR gates. The longest path to S is from A or B through axb

and then through the second XOR to S, giving a depth of 2 gate levels.



The output C_out is produced by “OR-ing” two carry terms. The longest path to C_out is from A or B through axb, then through the AND gate that forms cin_axb, and finally through the OR gate. This gives a depth of 3 gate levels. Other paths to Cout are shorter.



The full adder has two main input-to-output paths to consider: the sum path and the carry-out path. The longest path to the sum output passes from A or B through two successive XOR gates, first generating A ⊕ B and then combining it with Cin. With each XOR assigned a delay of 5 ns, this path takes **10 ns**. The longest path to carry-out passes through the first XOR, an AND gate that combines the propagate signal with Cin, and an OR gate that combines this term with A · B. With AND and OR

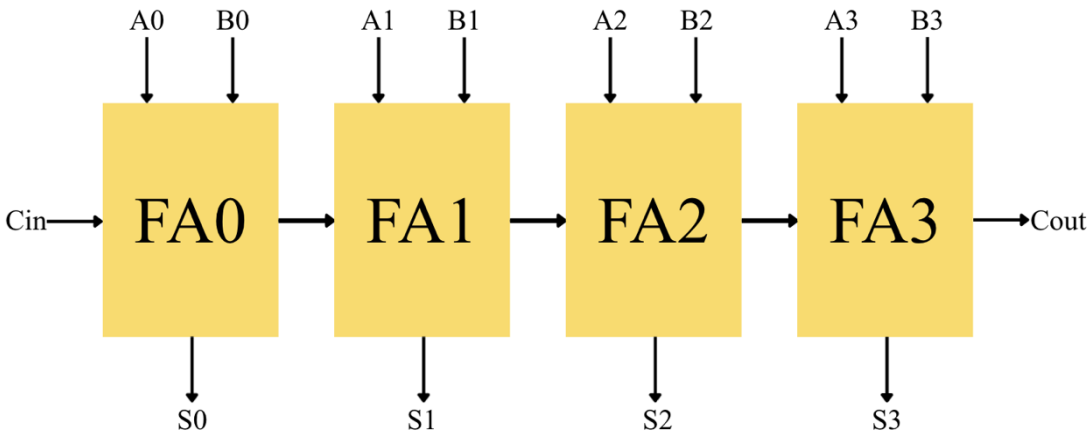
delays of 2 ns each, this path takes **5 + 2 + 2 = 9 ns**. Therefore, the longest path within an individual full adder is the 10 ns sum path.

This information is critical later when analyzing gate delay of different adder implementations

Phase 2: Constructing 4-Bit Adder Blocks

After implementing the 1-bit full adder, the next step is to build reusable 4-bit adder blocks. Each block accepts two 4-bit operands and a carry-in, then produces a 4-bit sum and a carry-out. Two architectures are implemented: ripple-carry and carry-lookahead.

For the ripple-carry block, four full adders are connected in sequence. The first stage receives the block's carry-in, and each subsequent stage receives the carry-out of the previous stage. This structure is simple, but the carry must propagate through successive stages before the final result settles.



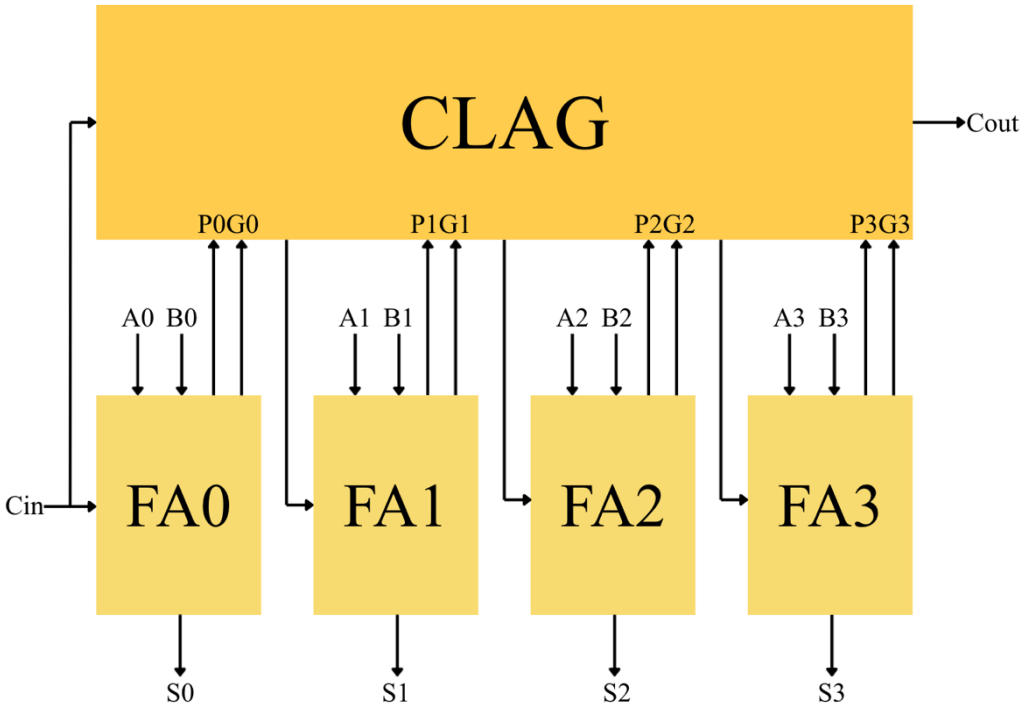
For the carry-lookahead block, each bit generates propagate and generate signals:

$P[i] = A[i] \oplus B[i]$
$G[i] = A[i] \cdot B[i]$

These signals allow the carry-lookahead generator to calculate carries directly from the inputs instead of waiting for each preceding stage. The block also produces group propagate, PG, and group generate, GG, which describe the carry behavior of all four

bits. These outputs support lookahead between blocks when constructing the 16-bit design.

Both block types expose the carry entering their most significant bit, C_msb. In the highest 4-bit block, this signal is later used for signed overflow detection.



Phase 3: Design of Addition and Subtraction Logic

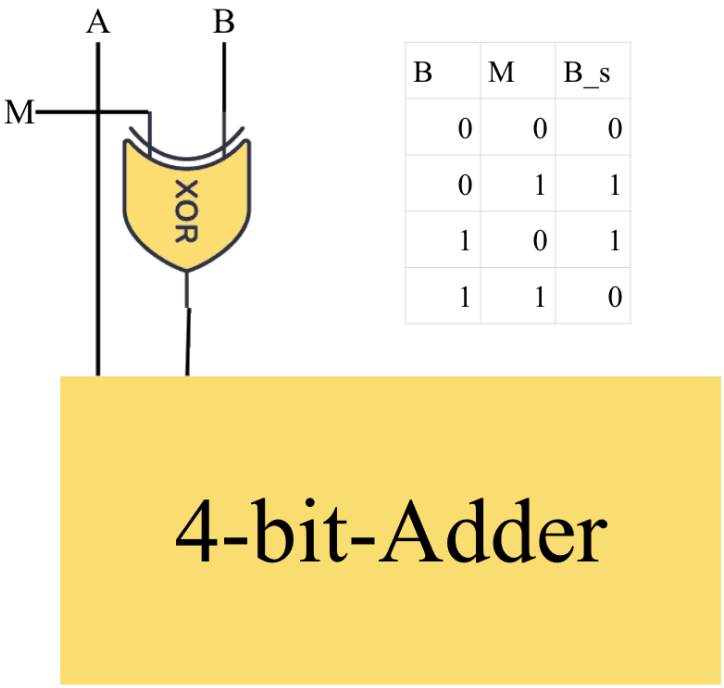
To support addition and subtraction using the same adder, the circuit uses a control signal, M, to select the operation. Each bit of B passes through an XOR gate controlled by M, and M also serves as the carry-in to the least significant adder block. The resulting operation is:

$\text{sum} = A + (B \oplus \{16\{M\}\}) + M$

When M = 0, the XOR gates pass B unchanged and the initial carry-in is 0. The circuit therefore performs A + B.

When M = 1, the XOR gates invert every bit of B and the initial carry-in becomes 1. The circuit performs A + ~B + 1, which is equivalent to A − B using two's-complement arithmetic. This approach allows both operations to share the same four 4-bit adder blocks.

The output sum contains the lower 16 bits of the result. During addition, c_out indicates a carry beyond bit 15. During subtraction, c_out = 1 indicates no unsigned borrow, while c_out = 0 indicates a borrow. Signed overflow is detected separately because carry-out alone does not indicate whether a signed result is valid.



B	M	B_s
0	0	0
0	1	1
1	0	1
1	1	0

When **M = 0**, B passes unchanged. When **M = 1**, B is inverted.

Phase 4: Integrating the 16-Bit Adder/Subtractor

Four 4-bit adder blocks form the 16-bit circuit. Mode M controls B inversion and the initial carry-in to select addition or subtraction. Carries ripple between RCA blocks or are calculated by lookahead logic in the CLA. The block outputs combine into sum[15:0] and c_out, while signed versions XOR the carries entering and leaving bit 15 to detect overflow.

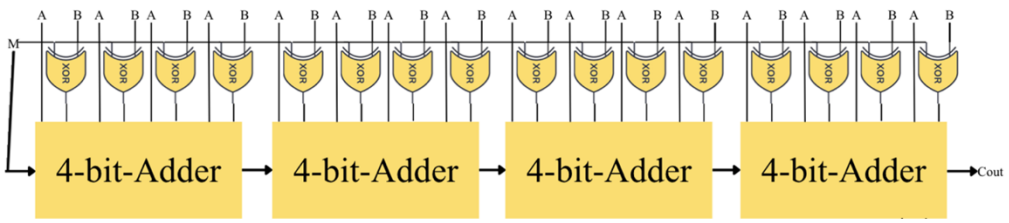
II. Circuit Architecture

i. Unsigned Ripple-Carry Adder/Subtractor

The unsigned RCA is constructed by connecting four 4-bit ripple-carry blocks. These blocks process bits [3:0], [7:4], [11:8], and [15:12] of A and B_selected. The lowest block receives M as its carry-in, and each block's carry-out connects to the next block's carry-in.

The four partial sums form the 16-bit sum output, while the highest block produces c_out. With M = 0, the circuit performs addition; with M = 1, it performs subtraction. This architecture is straightforward, but the carry may need to propagate through all 16 bit positions before the outputs settle.

ii. Signed Ripple-Carry Adder/Subtractor with Overflow



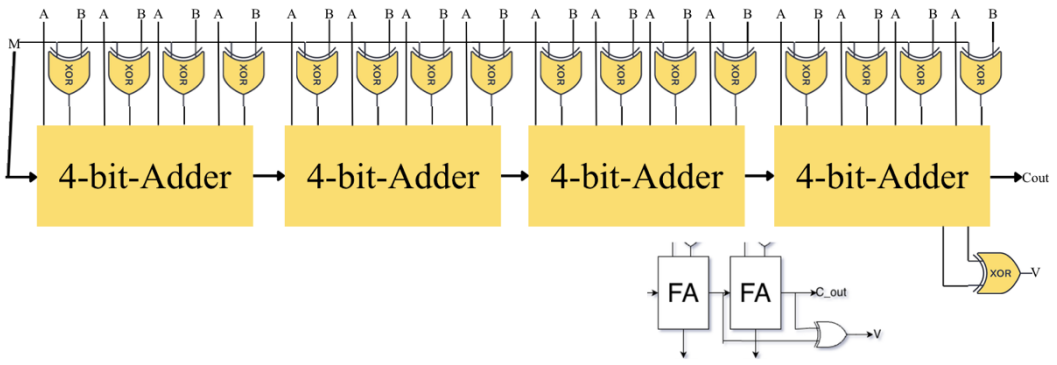
The signed RCA uses the same four-block structure and addition/subtraction logic as the unsigned RCA. The operands and result are interpreted as 16-bit two's-complement numbers, whose representable range is −32,768 to 32,767.

An additional XOR gate detects signed overflow by comparing the carry entering bit 15 with the carry leaving that bit. The incoming carry is provided by C_msb of the highest block:

overflow = C_msb[3] ⊕ c_out

If these carries differ, the mathematical result is outside the signed range. The sum output still contains the lower 16 bits of the result, while overflow indicates that the

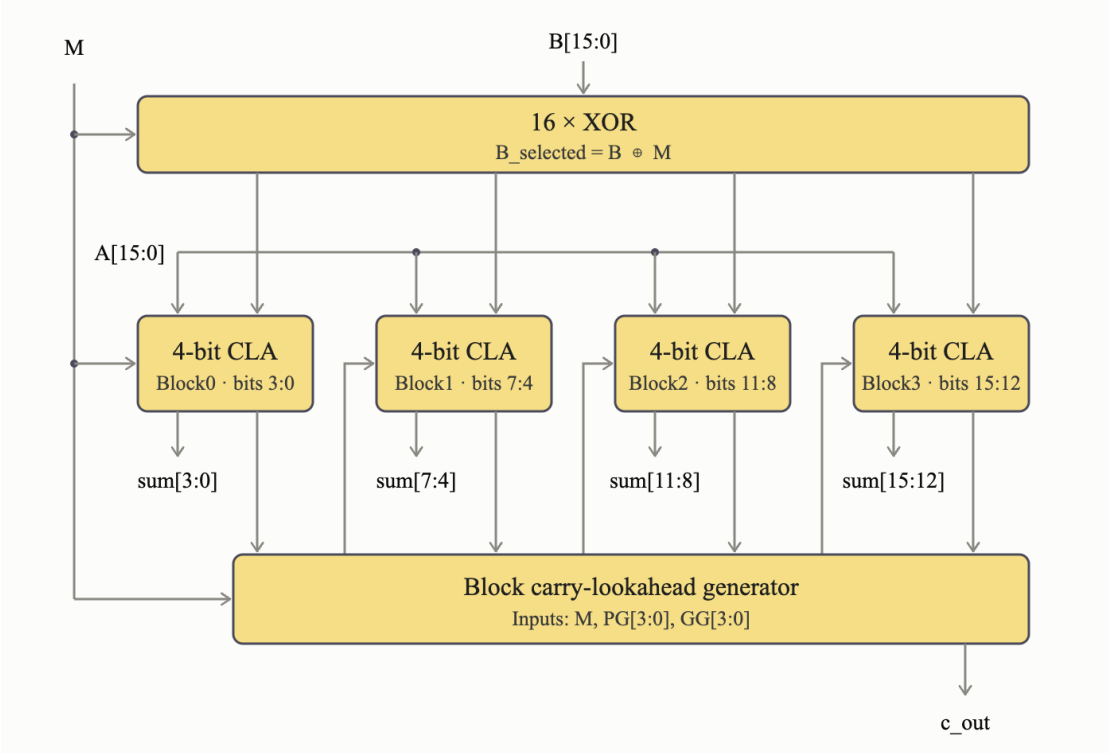
signed result cannot be represented correctly.



iii. Unsigned Carry-Lookahead Adder/Subtractor

The unsigned CLA combines four 4-bit carry-lookahead blocks. Each block computes its local sum and produces group propagate, PG, and group generate, GG, signals. A second-level carry-lookahead generator receives these group signals and M to calculate the carries entering the higher blocks and the final c_out.

This arrangement avoids waiting for carries to ripple sequentially through all four blocks. Each block uses its incoming carry and local propagate/generate signals to calculate its internal carries. The four partial sums are combined into the 16-bit sum output. The same mode-controlled B inversion supports both addition and subtraction.

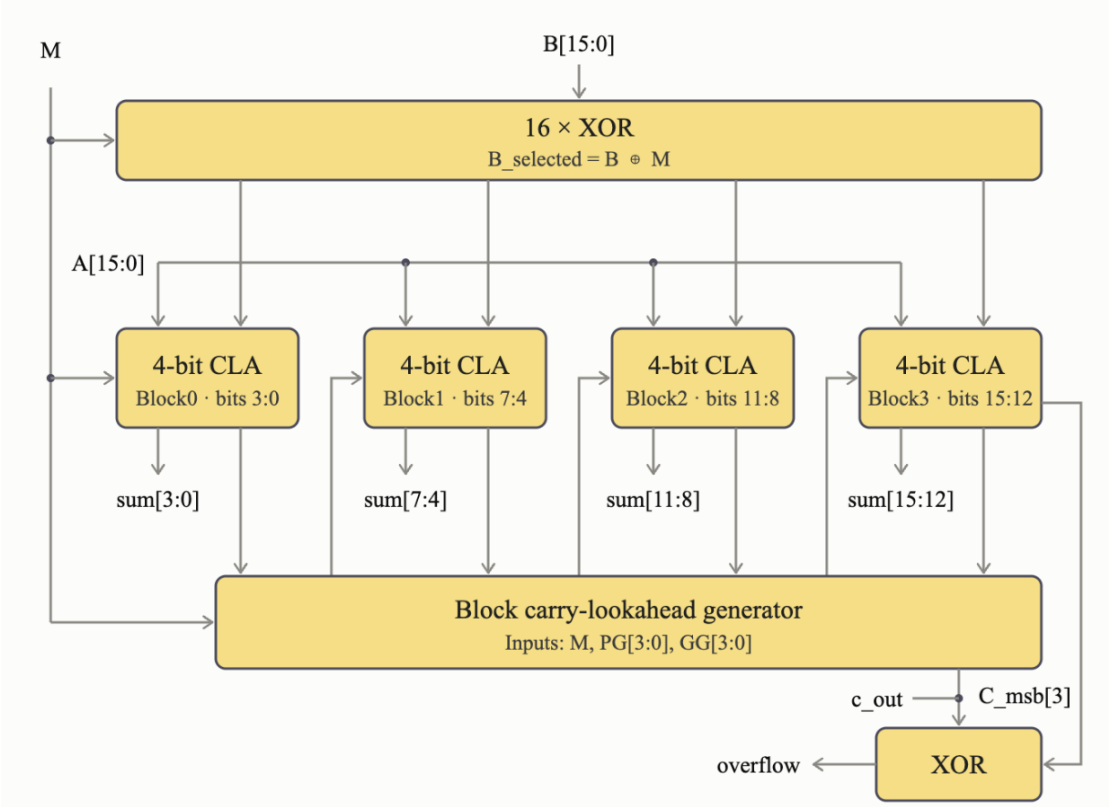


iv. Signed Carry-Lookahead Adder/Subtractor with Overflow

The signed CLA uses the same two-level lookahead architecture as the unsigned CLA, with an additional overflow-detection XOR gate. The highest 4-bit block provides the carry entering bit 15 through C_msb[3], while the second-level lookahead generator provides c_out.

overflow = C_msb[3] ⊕ c_out

This design combines faster carry calculation with signed overflow detection. Its arithmetic data path is identical to the unsigned CLA; the additional overflow output indicates when an addition or subtraction result falls outside the 16-bit signed range.



Overflow Logic

Signed overflow occurs when an addition or subtraction produces a result outside the 16-bit two's-complement range of −32,768 to 32,767. Both signed designs detect this by XORing the carry entering the most significant bit with the carry leaving it: overflow = C_msb[3] ⊕ c_out. Here, C_msb[3] is the carry entering bit 15 within the highest 4-bit block. If these two carries differ, overflow is asserted, indicating that the 16-bit sum cannot correctly represent the signed mathematical result. This method works for both addition and two's-complement subtraction and requires only one

additional XOR gate. Carry-out alone cannot indicate signed overflow; it instead represents unsigned carry or, during subtraction, the absence of a borrow.

III. Design Comparison

Gate Count and Optimization

For this stage of the design, assume the CLA uses standard full adders that provide sum, carry-out, propagate, and generate signals. The propagate and generate signals reuse the full adder's existing XOR and AND outputs, so they require no additional gates.

Each full adder contains **two XOR gates, two AND gates, and one OR gate**, giving five gates per bit.

For the **RCA**, four full adders form a 4-bit block containing 20 gates. Four blocks require 80 gates, and the 16 mode-selection XOR gates increase the total to **96 gates**. Adding signed overflow detection gives **97 gates**.

For the **CLA**, each 4-bit block contains four full adders, a local carry-lookahead generator, and group propagate/generate logic:

Component	XOR	AND	OR	Total
Four full	8	8	4	20
Local	0	10	4	14
Group	0	4	1	5
4-bit CLA	**8**	**22**	**9**	**39**

Four CLA blocks therefore require 156 gates. The second-level carry-lookahead generator adds 14 gates, and mode selection adds 16 XOR gates. This gives **186 gates**

for the unsigned CLA and **187 gates** for the signed CLA with overflow detection.

Design	XOR	AND	OR	Total
Unsigned RCA	48	32	16	**96**
Signed RCA	49	32	16	**97**
Unsigned CLA	48	98	40	**186**
Signed CLA	49	98	40	**187**

Under this implementation, the CLA uses **90 more gates** than the corresponding RCA. The additional carry-lookahead logic reduces the sequential carry dependency at the cost of increased hardware complexity. Each signed version requires only one extra XOR gate for overflow detection.

These counts treat each multi-input primitive as one gate and include the full adders' unused carry-out circuitry before optimization.

The next improvement is to replace the standard full adders in the CLA with **no_Cout_adder cells**. In the CLA architecture, carry signals are already calculated by a separate carry-lookahead generator. Therefore, the individual full adders' carry-out circuitry is redundant.

A standard full adder calculates:

$P = A \oplus B$
$G = A \cdot B$
$S = P \oplus Cin$
$Cout = G + P \cdot Cin$

The no_Cout_adder retains the sum, propagate, and generate outputs but removes the local Cout calculation. This eliminates the AND gate used for $P \cdot Cin$ and the OR gate used to combine it with G.

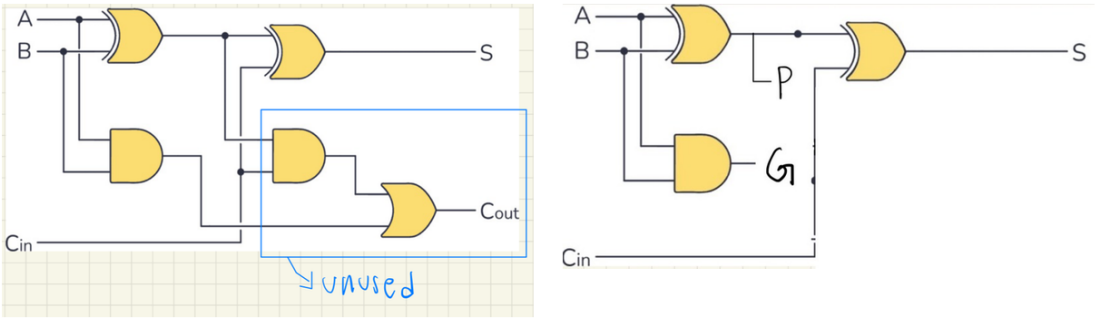
Cell type	XOR	AND	OR	Total
Standard full	2	2	1	5
no_Cout_adder	2	1	0	3

Each replacement saves **two gates**. Since the 16-bit CLA contains sixteen cells, the total reduction is **32 gates: 16 AND gates and 16 OR gates**.

CLA design	Using full adders	Using no_Cout_adder	Gates saved
Unsigned	186	154	32
Signed with overflow	187	155	32

This modification preserves the arithmetic behavior and the carry-lookahead structure. The sum cells still provide P and G to the carry generators and use the generated carries to calculate their sums. The critical path to the required outputs is unchanged because the removed local carry-out circuitry was unused.

These counts describe the instantiated gate primitives before synthesis optimization. The separate carry-out logic within each 4-bit CLA block remains included in the totals.



Possible Improvements

One possible improvement is to remove redundant carry-generation logic from the CLA blocks. In the original design, each 4-bit block calculates four carry outputs, but its final carry-out is unused because the second-level Block_CLAG already calculates the carries between blocks. Therefore, each local four-output carry-lookahead generator can be replaced with a three-output generator that produces only the carries needed inside the block.

This modification removes four AND gates and one OR gate per block, saving **20 gates across the four blocks**. The unsigned CLA gate count decreases from **154 to 134**, while the signed CLA decreases from **155 to 135**. The group propagate and group generate logic remains unchanged, as does the four-output Block_CLAG. Since

only unused logic is removed, the arithmetic behavior and estimated critical-path delay remain unchanged.

The reduction describes the gate primitives explicitly instantiated in the source; synthesis tools may already eliminate the unused logic automatically.

Longest Critical Path

The longest critical path determines the maximum estimated time required for the outputs to settle after an input change. To compare the four architectures, we assume **5 ns for XOR gates, 2 ns for AND/OR gates, and 1 ns for inverters**. Each multi-input gate is treated as one gate level. The mode-selection logic is implemented using XOR gates, so its delay is also 5 ns.

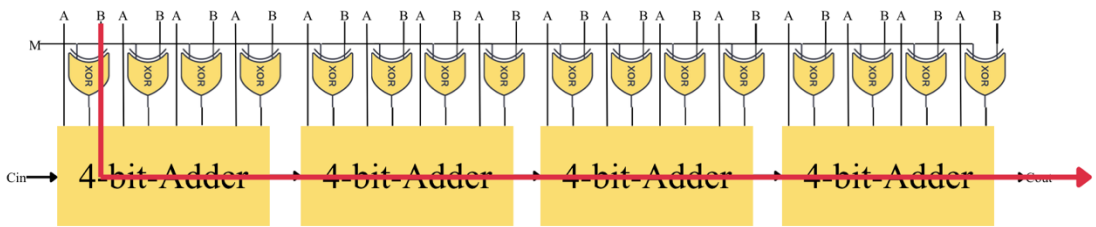
These estimates use a uniform delay model. Any AND/OR gates still set to #1 in the code must be changed to #2 for simulation to match this analysis.

Unsigned RCA

The longest structural path passes through the mode-selection XOR, the propagate XOR of the first full adder, and fifteen carry stages, each containing an AND gate followed by an OR gate. It then passes through the final sum XOR to produce sum[15].

Longest path ex: A=0000 B=0001 → 0000 M=1

First let A=0000, B=0001, M=1 settle, then change only B[0] from 1 to 0. The mode-selection XOR converts this into a rising transition on B_selected[0]. It passes through the first propagate XOR and launches a carry. Every higher bit propagates that carry through its AND–OR logic, across all four RCA blocks. The final XOR produces sum[15], giving an estimated delay of **75 ns**.



Estimated delay = 5 + 5 + 15 × (2 + 2) + 5
= 75 ns

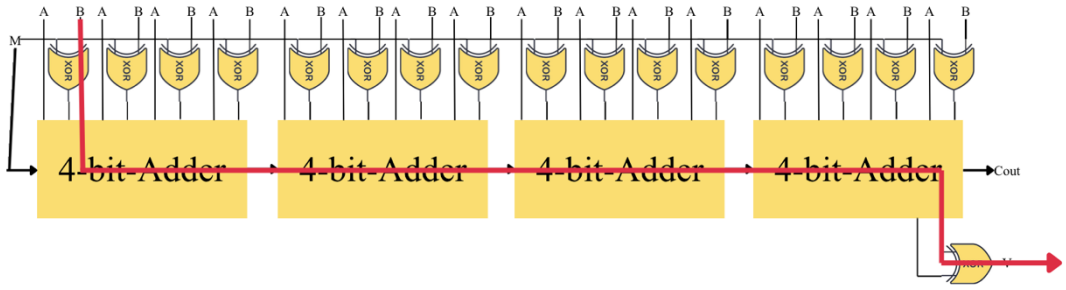
The longest path contains 34 gate levels. Although the circuit is organized into four 4-bit blocks, the carry still propagates sequentially through all sixteen stages.

Signed RCA with Overflow

The signed RCA uses the same ripple-carry structure and adds an XOR gate to compare the carry entering bit 15 with the final carry-out. The longest structural path extends through the carry chain and then through the overflow XOR.

Longest path ex: A=0000 B=8001 → 8000 M=1

Change only B[0] from 1 to 0 after the initial inputs settle. The transition passes through the mode-selection and propagate XORs, then ripples through bits 0–14 to the carry entering bit 15. Bit 15 keeps c_out=0, so the incoming carry changes the overflow output through the final XOR. The operation changes from 0−(−32767) to 0−(−32768), causing V to change from 0 to 1 after an estimated **75 ns**. This approximately activates the longest path; the **79 ns structural bound through c_out is not demonstrated by this transition**.



Estimated delay = 5 + 5 + 16 × (2 + 2) + 5
= 79 ns

This path contains 35 gate levels. Overflow detection adds 5 ns to the structural upper-bound estimate. Its observed waveform may include temporary glitches because the two carry inputs can arrive at different times.

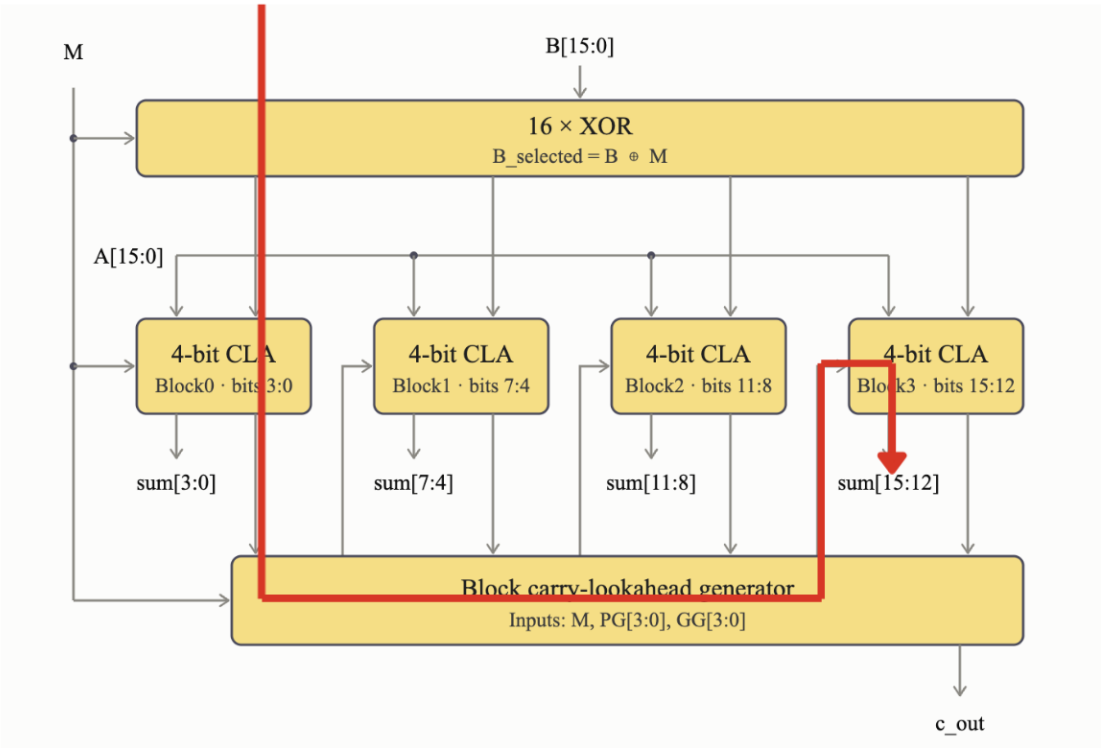
Unsigned CLA

The longest structural path passes through mode selection, bit-propagate generation, group-generate logic, the second-level carry-lookahead generator, and the destination block's local carry-lookahead generator. A final XOR produces the sum bit.

Under this delay model, the propagate XOR is slower than the bit-generate AND gate, so the longest path begins through propagate generation.

Longest path ex: A=0001 B=0002 → 0000 M=1

Change only B[1] from 1 to 0. Bit 0 already generates a carry, and the change at bit 1 allows that carry to propagate through the lowest block. The signal travels through the mode-selection XOR, bit-propagate XOR, and Block 0's group-generate logic. Block_CLAG then calculates the carry entering Block 3 directly. Block 3's local lookahead logic calculates the carry entering bit 15, and the final sum XOR changes sum[15]. The estimated delay is **27 ns**.



B → mode-selection XOR → propagate XOR
→ group-generate AND/OR
→ second-level lookahead AND/OR
→ local lookahead AND/OR → sum XOR

Estimated delay = 5 + 5 + (2 + 2) + (2 + 2) + (2 + 2) + 5
= 27 ns

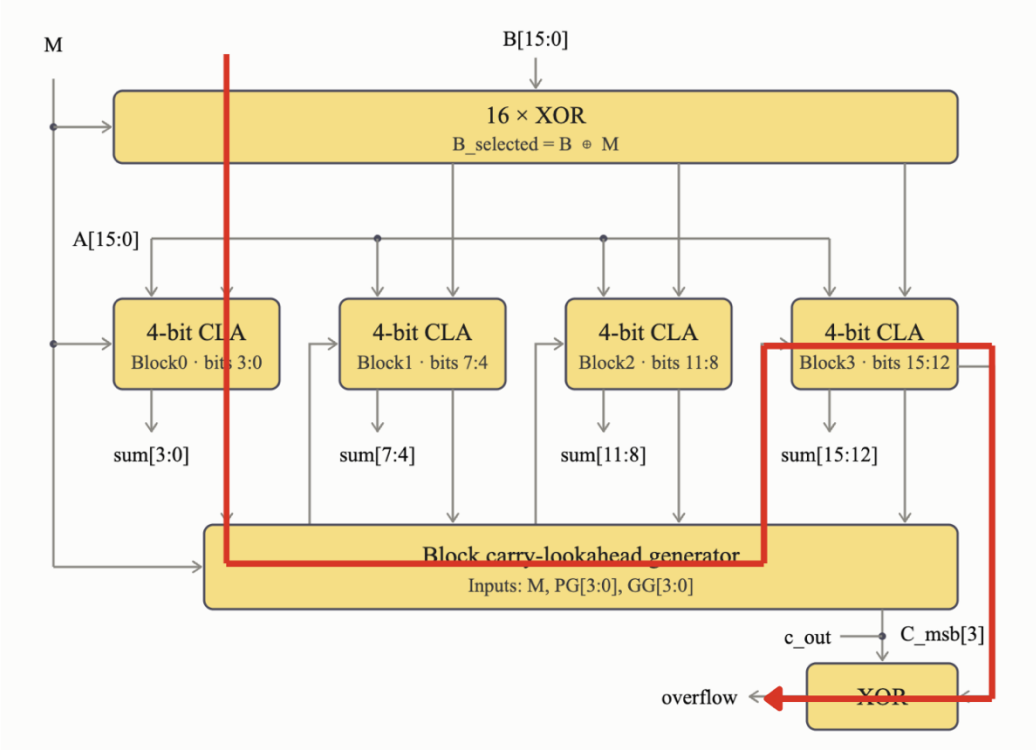
This path contains nine gate levels. Carry lookahead reduces the delay by calculating carries through parallel logic rather than a sixteen-stage ripple chain.

Signed CLA with Overflow

The signed CLA adds an XOR gate that compares the carry entering bit 15 with c_out. The longest path to overflow passes through the local lookahead logic producing that incoming carry, followed by the overflow XOR.

Longest path ex: A=0001 B=8002 → 8000 M=1

Change only B[1] from 1 to 0. The transition follows the same propagate, group-generate, and two-level lookahead path to the carry entering bit 15. Here, c_out remains 0, allowing that carry transition to pass through the overflow XOR. The operation changes from 1−(−32766)=32767 to 1−(−32768)=32769, so V changes from 0 to 1. The estimated delay is **27 ns**.



B → mode-selection XOR → propagate XOR
→ group-generate AND/OR
→ second-level lookahead AND/OR
→ local lookahead AND/OR → overflow XOR

Estimated delay = 5 + 5 + (2 + 2) + (2 + 2) + (2 + 2) + 5

= 27 ns

The overflow XOR operates in parallel with the final sum XOR rather than following it. Therefore, the maximum structural depth remains nine gate levels, and the estimated maximum delay remains 27 ns.

Design	Longest structural path	Estimated maximum delay
Unsigned RCA	34 gate levels	**74 ns**
Signed RCA with overflow	35 gate levels	**79 ns**
Unsigned CLA	9 gate levels	**27 ns**
Signed CLA with overflow	9 gate levels	**27 ns**

These values are structural upper-bound estimates. A simulation waveform must be used to establish the actual propagation delay for a selected input transition.

Conclusion

The four designs demonstrate the tradeoff between hardware complexity and propagation delay. The unsigned and signed RCA designs require 96 and 97 gates, respectively, with estimated maximum delays of 74 ns and 79 ns. Using no_Cout_adder cells, the unsigned and signed CLA designs require 154 and 155 gates but reduce the estimated maximum delay to 27 ns through two-level carry lookahead. Signed overflow detection adds only one XOR gate to either architecture. It extends the RCA's longest structural path, while the CLA's overflow calculation runs in parallel with the final sum calculation and therefore does not increase its maximum delay. Overall, the RCA offers a simpler circuit with fewer gates, whereas the CLA uses additional logic to achieve faster operation.

IV. Testbench Design

The testbench design initially aimed to verify every possible input combination. With two 16-bit operands and two operation modes, exhaustive testing would require 8,589,934,592 cases for each circuit. Because gate-level simulation includes propagation delays and many internal signal transitions, completing this many tests would take considerable time. The exhaustive approach was therefore replaced with 10,000 repeatable random cases, 2,048 block-sweep cases, and 26 selected cases, giving **12,074 checks per design**. The selected cases cover normal arithmetic, carry-out, borrow, signed overflow, identity operations, and carry propagation across 4-bit block boundaries. A fixed random seed makes the test sequence reproducible.

Two testbench variants are used: one for unsigned designs and one for signed designs with overflow detection. Both calculate an expected sum and carry-out using behavioral arithmetic, then compare these values against the circuit outputs. During subtraction, the expected carry-out is determined by whether A is greater than or equal to B, since carry-out indicates the absence of an unsigned borrow. The signed testbench additionally connects the overflow output and checks it against an independent reference calculation: the operands are interpreted as signed integers, and overflow is expected when the mathematical result falls outside −32,768 to 32,767. Case-inequality comparisons (!==) also flag unknown or high-impedance outputs as failures.

For the same arithmetic variant, the RCA and CLA testbenches use the same inputs, reference calculations, checking procedures, and output format. The only difference is the settling time before checking the outputs: **80 ns for RCA and 32 ns for CLA**. These intervals exceed their estimated maximum propagation delays, preventing a correct circuit from being reported as faulty while its outputs are still changing. Each test then includes a further 10 ns hold interval. The terminal output reports failures, displays selected cases in aligned columns with simulation time, and finishes with the total number of checks, error count, and PASS or FAIL result.

Below is the terminal output for each design

Unsigned RCA:

```
This is a very awesome testbench designed by Bingo
beginning test.............
Each case is held for 10ns for clear waveform time frames.

Testing.................
[RANDOM CASES]_____________________________________________________________________________________________________
RANDOM SUMMARY: 12048 checks, 0 errors
[SELECTED CASES]-----------------------------------------------------------------------------------------------------
|   Time(ns) | TEST | M |  A   |  B   | SUM  | C_out | EXP_SUM | EXP_C | Description
|    1084400 | PASS | 0 | 1234 | 4321 | 5555 |   0   | 5555    |   0   | ADD basic
|    1084490 | PASS | 0 | 0000 | 0000 | 0000 |   0   | 0000    |   0   | ADD zero
|    1084580 | PASS | 0 | ffff | 0001 | 0000 |   1   | 0000    |   1   | ADD carry out
|    1084670 | PASS | 0 | ffff | ffff | fffe |   1   | fffe    |   1   | ADD max plus max
|    1084760 | PASS | 0 | 7fff | 0001 | 8000 |   0   | 8000    |   0   | ADD positive signed overflow
|    1084850 | PASS | 0 | 8000 | 8000 | 0000 |   1   | 0000    |   1   | ADD negative signed overflow
|    1084940 | PASS | 1 | 4321 | 1234 | 30ed |   1   | 30ed    |   1   | SUB basic
|    1085030 | PASS | 1 | 0000 | 0001 | ffff |   0   | ffff    |   0   | SUB borrow
|    1085120 | PASS | 1 | 0000 | ffff | 0001 |   0   | 0001    |   0   | SUB unsigned underflow edge
|    1085210 | PASS | 1 | 8000 | 0001 | 7fff |   1   | 7fff    |   1   | SUB negative signed overflow
|    1085300 | PASS | 1 | 7fff | ffff | 8000 |   0   | 8000    |   0   | SUB positive signed overflow
|    1085390 | PASS | 0 | a5a5 | 0000 | a5a5 |   0   | a5a5    |   0   | ADD identity
|    1085480 | PASS | 1 | a5a5 | 0000 | a5a5 |   1   | a5a5    |   1   | SUB identity
|    1085570 | PASS | 1 | 5c5c | 5c5c | 0000 |   1   | 0000    |   1   | SUB self
|    1085660 | PASS | 0 | 5555 | aaaa | ffff |   0   | ffff    |   0   | ADD alternating bits
|    1085750 | PASS | 1 | aaaa | 5555 | 5555 |   1   | 5555    |   1   | SUB alternating bits
|    1085840 | PASS | 0 | 000f | 0001 | 0010 |   0   | 0010    |   0   | ADD across block boundary
|    1085930 | PASS | 1 | 0010 | 0001 | 000f |   1   | 000f    |   1   | SUB across block boundary
|    1086020 | PASS | 0 | 00ff | 0001 | 0100 |   0   | 0100    |   0   | ADD across block boundary
|    1086110 | PASS | 1 | 0100 | 0001 | 00ff |   1   | 00ff    |   1   | SUB across block boundary
|    1086200 | PASS | 0 | 0fff | 0001 | 1000 |   0   | 1000    |   0   | ADD across block boundary
|    1086290 | PASS | 1 | 1000 | 0001 | 0fff |   1   | 0fff    |   1   | SUB across block boundary
|    1086380 | PASS | 0 | ffff | 0000 | ffff |   0   | ffff    |   0   | Carry propagation baseline
|    1086470 | PASS | 0 | ffff | 0001 | 0000 |   1   | 0000    |   1   | Carry propagation transition
|    1086560 | PASS | 0 | 5aa5 | 1234 | 6cd9 |   0   | 6cd9    |   0   | Mode transition add
|    1086650 | PASS | 1 | 5aa5 | 1234 | 4871 |   1   | 4871    |   1   | Mode transition subtract
SUMMARY: 12074 checks, 0 errors
RESULT: PASS
tb.v:134: $finish called at 1086660000 (1ps)
```

Signed RCA with Overflow:

```
Testing.................
[RANDOM CASES]_____________________________________________________________________________________________________
RANDOM SUMMARY: 12048 checks, 0 errors
[SELECTED CASES]-----------------------------------------------------------------------------------------------------
|   Time(ns) | TEST | M |  A   |  B   | SUM  | C_out |   V   | EXP_SUM | EXP_C | EXP_V | Description
|    1084400 | PASS | 0 | 1234 | 4321 | 5555 |   0   |   0   | 5555    |   0   |   0   | ADD basic
|    1084490 | PASS | 0 | 0000 | 0000 | 0000 |   0   |   0   | 0000    |   0   |   0   | ADD zero
|    1084580 | PASS | 0 | ffff | 0001 | 0000 |   1   |   0   | 0000    |   1   |   0   | ADD carry out
|    1084670 | PASS | 0 | ffff | ffff | fffe |   1   |   0   | fffe    |   1   |   0   | ADD max plus max
|    1084760 | PASS | 0 | 7fff | 0001 | 8000 |   0   |   1   | 8000    |   0   |   1   | ADD positive signed overflow
|    1084850 | PASS | 0 | 8000 | 8000 | 0000 |   1   |   1   | 0000    |   1   |   1   | ADD negative signed overflow
|    1084940 | PASS | 1 | 4321 | 1234 | 30ed |   1   |   0   | 30ed    |   1   |   0   | SUB basic
|    1085030 | PASS | 1 | 0000 | 0001 | ffff |   0   |   0   | ffff    |   0   |   0   | SUB borrow
|    1085120 | PASS | 1 | 0000 | ffff | 0001 |   0   |   0   | 0001    |   0   |   0   | SUB unsigned underflow edge
|    1085210 | PASS | 1 | 8000 | 0001 | 7fff |   1   |   1   | 7fff    |   1   |   1   | SUB negative signed overflow
|    1085300 | PASS | 1 | 7fff | ffff | 8000 |   0   |   1   | 8000    |   0   |   1   | SUB positive signed overflow
|    1085390 | PASS | 0 | a5a5 | 0000 | a5a5 |   0   |   0   | a5a5    |   0   |   0   | ADD identity
|    1085480 | PASS | 1 | a5a5 | 0000 | a5a5 |   1   |   0   | a5a5    |   1   |   0   | SUB identity
|    1085570 | PASS | 1 | 5c5c | 5c5c | 0000 |   1   |   0   | 0000    |   1   |   0   | SUB self
|    1085660 | PASS | 0 | 5555 | aaaa | ffff |   0   |   0   | ffff    |   0   |   0   | ADD alternating bits
|    1085750 | PASS | 1 | aaaa | 5555 | 5555 |   1   |   1   | 5555    |   1   |   1   | SUB alternating bits
|    1085840 | PASS | 0 | 000f | 0001 | 0010 |   0   |   0   | 0010    |   0   |   0   | ADD across block boundary
|    1085930 | PASS | 1 | 0010 | 0001 | 000f |   1   |   0   | 000f    |   1   |   0   | SUB across block boundary
|    1086020 | PASS | 0 | 00ff | 0001 | 0100 |   0   |   0   | 0100    |   0   |   0   | ADD across block boundary
|    1086110 | PASS | 1 | 0100 | 0001 | 00ff |   1   |   0   | 00ff    |   1   |   0   | SUB across block boundary
|    1086200 | PASS | 0 | 0fff | 0001 | 1000 |   0   |   0   | 1000    |   0   |   0   | ADD across block boundary
|    1086290 | PASS | 1 | 1000 | 0001 | 0fff |   1   |   0   | 0fff    |   1   |   0   | SUB across block boundary
|    1086380 | PASS | 0 | ffff | 0000 | ffff |   0   |   0   | ffff    |   0   |   0   | Carry propagation baseline
|    1086470 | PASS | 0 | ffff | 0001 | 0000 |   1   |   0   | 0000    |   1   |   0   | Carry propagation transition
|    1086560 | PASS | 0 | 5aa5 | 1234 | 6cd9 |   0   |   0   | 6cd9    |   0   |   0   | Mode transition add
|    1086650 | PASS | 1 | 5aa5 | 1234 | 4871 |   1   |   0   | 4871    |   1   |   0   | Mode transition subtract
SUMMARY: 12074 checks, 0 errors
RESULT: PASS
tb.v:135: $finish called at 1086660000 (1ps)
```

Signed CLA with Overflow:

```
Testing..................
[RANDOM CASES]-----------------------------------------------------------------------------------------------------------
RANDOM SUMMARY: 12048 checks, 0 errors
[SELECTED CASES]---------------------------------------------------------------------------------------------------------
|    Time(ns) | TEST | M |   A    |   B    | SUM  | C_out | EXP_SUM | EXP_C | Description
|      506048 | PASS | 0 | 1234 | 4321 | 5555 |   0   | 5555    |   0   | ADD basic
|      506090 | PASS | 0 | 0000 | 0000 | 0000 |   0   | 0000    |   0   | ADD zero
|      506132 | PASS | 0 | ffff | 0001 | 0000 |   1   | 0000    |   1   | ADD carry out
|      506174 | PASS | 0 | ffff | ffff | fffe |   1   | fffe    |   1   | ADD max plus max
|      506216 | PASS | 0 | 7fff | 0001 | 8000 |   0   | 8000    |   0   | ADD positive signed overflow
|      506258 | PASS | 0 | 8000 | 8000 | 0000 |   1   | 0000    |   1   | ADD negative signed overflow
|      506300 | PASS | 1 | 4321 | 1234 | 30ed |   1   | 30ed    |   1   | SUB basic
|      506342 | PASS | 1 | 0000 | 0001 | ffff |   0   | ffff    |   0   | SUB borrow
|      506384 | PASS | 1 | 0000 | ffff | 0001 |   0   | 0001    |   0   | SUB unsigned underflow edge
|      506426 | PASS | 1 | 8000 | 0001 | 7fff |   1   | 7fff    |   1   | SUB negative signed overflow
|      506468 | PASS | 1 | 7fff | ffff | 8000 |   0   | 8000    |   0   | SUB positive signed overflow
|      506510 | PASS | 0 | a5a5 | 0000 | a5a5 |   0   | a5a5    |   0   | ADD identity
|      506552 | PASS | 1 | a5a5 | 0000 | a5a5 |   1   | a5a5    |   1   | SUB identity
|      506594 | PASS | 1 | 5c5c | 5c5c | 0000 |   1   | 0000    |   1   | SUB self
|      506636 | PASS | 0 | 5555 | aaaa | ffff |   0   | ffff    |   0   | ADD alternating bits
|      506678 | PASS | 1 | aaaa | 5555 | 5555 |   1   | 5555    |   1   | SUB alternating bits
|      506720 | PASS | 0 | 000f | 0001 | 0010 |   0   | 0010    |   0   | ADD across block boundary
|      506762 | PASS | 1 | 0010 | 0001 | 000f |   1   | 000f    |   1   | SUB across block boundary
|      506804 | PASS | 0 | 00ff | 0001 | 0100 |   0   | 0100    |   0   | ADD across block boundary
|      506846 | PASS | 1 | 0100 | 0001 | 00ff |   1   | 00ff    |   1   | SUB across block boundary
|      506888 | PASS | 0 | 0fff | 0001 | 1000 |   0   | 1000    |   0   | ADD across block boundary
|      506930 | PASS | 1 | 1000 | 0001 | 0fff |   1   | 0fff    |   1   | SUB across block boundary
|      506972 | PASS | 0 | ffff | 0000 | ffff |   0   | ffff    |   0   | Carry propagation baseline
|      507014 | PASS | 0 | ffff | 0001 | 0000 |   1   | 0000    |   1   | Carry propagation transition
|      507056 | PASS | 0 | 5aa5 | 1234 | 6cd9 |   0   | 6cd9    |   0   | Mode transition add
|      507098 | PASS | 1 | 5aa5 | 1234 | 4871 |   1   | 4871    |   1   | Mode transition subtract
SUMMARY: 12074 checks, 0 errors
RESULT: PASS
tb.v:132: $finish called at 507108000 (1ps)
```

\

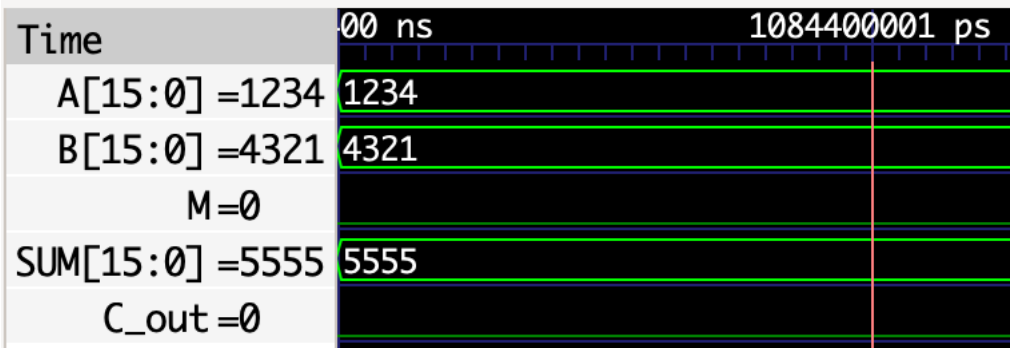
Signed CLA with Overflow:

```
Testing..................
[RANDOM CASES]-----------------------------------------------------------------------------------------------------------
RANDOM SUMMARY: 12048 checks, 0 errors
[SELECTED CASES]---------------------------------------------------------------------------------------------------------
|    Time(ns) | TEST | M |   A    |   B    | SUM  | C_out |    V    | EXP_SUM | EXP_C | EXP_V | Description
|      506048 | PASS | 0 | 1234 | 4321 | 5555 |   0   |    0    | 5555    |   0   |   0   | ADD basic
|      506090 | PASS | 0 | 0000 | 0000 | 0000 |   0   |    0    | 0000    |   0   |   0   | ADD zero
|      506132 | PASS | 0 | ffff | 0001 | 0000 |   1   |    0    | 0000    |   1   |   0   | ADD carry out
|      506174 | PASS | 0 | ffff | ffff | fffe |   1   |    0    | fffe    |   1   |   0   | ADD max plus max
|      506216 | PASS | 0 | 7fff | 0001 | 8000 |   0   |    1    | 8000    |   0   |   1   | ADD positive signed overflow
|      506258 | PASS | 0 | 8000 | 8000 | 0000 |   1   |    1    | 0000    |   1   |   1   | ADD negative signed overflow
|      506300 | PASS | 1 | 4321 | 1234 | 30ed |   1   |    0    | 30ed    |   1   |   0   | SUB basic
|      506342 | PASS | 1 | 0000 | 0001 | ffff |   0   |    0    | ffff    |   0   |   0   | SUB borrow
|      506384 | PASS | 1 | 0000 | ffff | 0001 |   0   |    0    | 0001    |   0   |   0   | SUB unsigned underflow edge
|      506426 | PASS | 1 | 8000 | 0001 | 7fff |   1   |    1    | 7fff    |   1   |   1   | SUB negative signed overflow
|      506468 | PASS | 1 | 7fff | ffff | 8000 |   0   |    1    | 8000    |   0   |   1   | SUB positive signed overflow
|      506510 | PASS | 0 | a5a5 | 0000 | a5a5 |   0   |    0    | a5a5    |   0   |   0   | ADD identity
|      506552 | PASS | 1 | a5a5 | 0000 | a5a5 |   1   |    0    | a5a5    |   1   |   0   | SUB identity
|      506594 | PASS | 1 | 5c5c | 5c5c | 0000 |   1   |    0    | 0000    |   1   |   0   | SUB self
|      506636 | PASS | 0 | 5555 | aaaa | ffff |   0   |    0    | ffff    |   0   |   0   | ADD alternating bits
|      506678 | PASS | 1 | aaaa | 5555 | 5555 |   1   |    1    | 5555    |   1   |   1   | SUB alternating bits
|      506720 | PASS | 0 | 000f | 0001 | 0010 |   0   |    0    | 0010    |   0   |   0   | ADD across block boundary
|      506762 | PASS | 1 | 0010 | 0001 | 000f |   1   |    0    | 000f    |   1   |   0   | SUB across block boundary
|      506804 | PASS | 0 | 00ff | 0001 | 0100 |   0   |    0    | 0100    |   0   |   0   | ADD across block boundary
|      506846 | PASS | 1 | 0100 | 0001 | 00ff |   1   |    0    | 00ff    |   1   |   0   | SUB across block boundary
|      506888 | PASS | 0 | 0fff | 0001 | 1000 |   0   |    0    | 1000    |   0   |   0   | ADD across block boundary
|      506930 | PASS | 1 | 1000 | 0001 | 0fff |   1   |    0    | 0fff    |   1   |   0   | SUB across block boundary
|      506972 | PASS | 0 | ffff | 0000 | ffff |   0   |    0    | ffff    |   0   |   0   | Carry propagation baseline
|      507014 | PASS | 0 | ffff | 0001 | 0000 |   1   |    0    | 0000    |   1   |   0   | Carry propagation transition
|      507056 | PASS | 0 | 5aa5 | 1234 | 6cd9 |   0   |    0    | 6cd9    |   0   |   0   | Mode transition add
|      507098 | PASS | 1 | 5aa5 | 1234 | 4871 |   1   |    0    | 4871    |   1   |   0   | Mode transition subtract
SUMMARY: 12074 checks, 0 errors
RESULT: PASS
tb.v:133: $finish called at 507108000 (1ps)
```

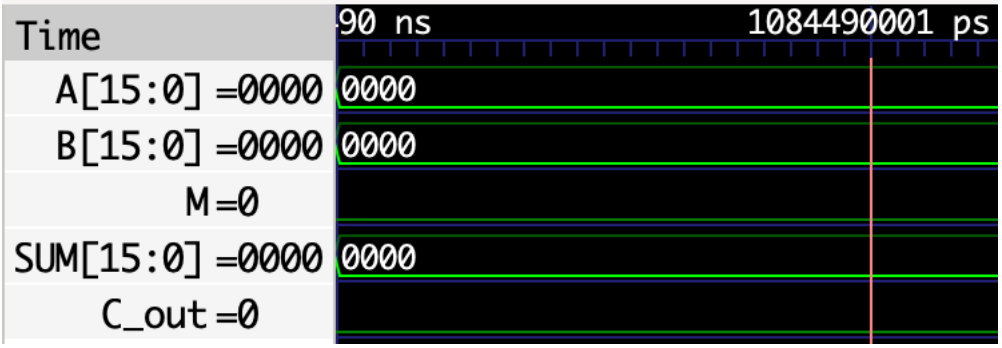
V. Waveform Analysis

Unsigned:

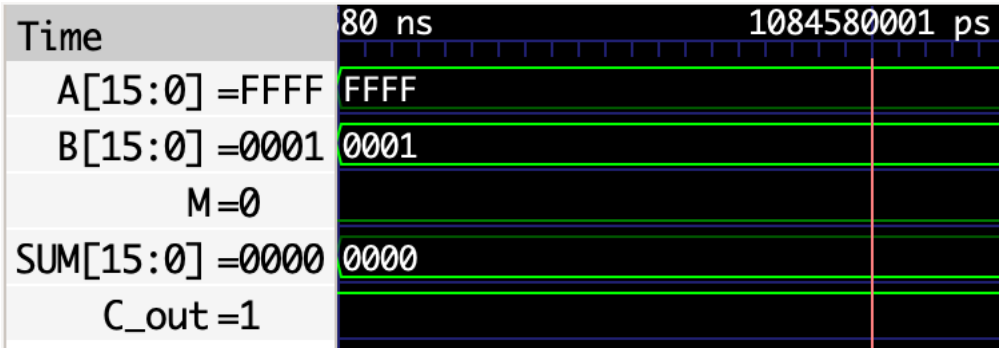
(ADD basic): A=1234 B=4321 M=0 SUM=5555 C_out=0



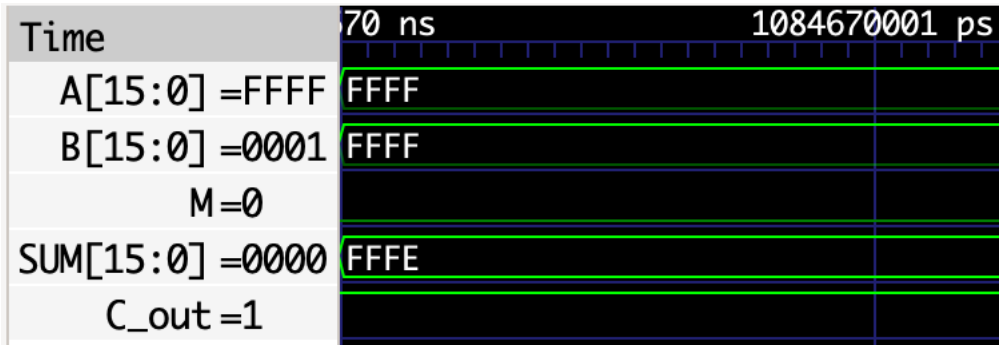
(ADD zero): A=0000 B=0000 M=0 SUM=0000 C_out=0



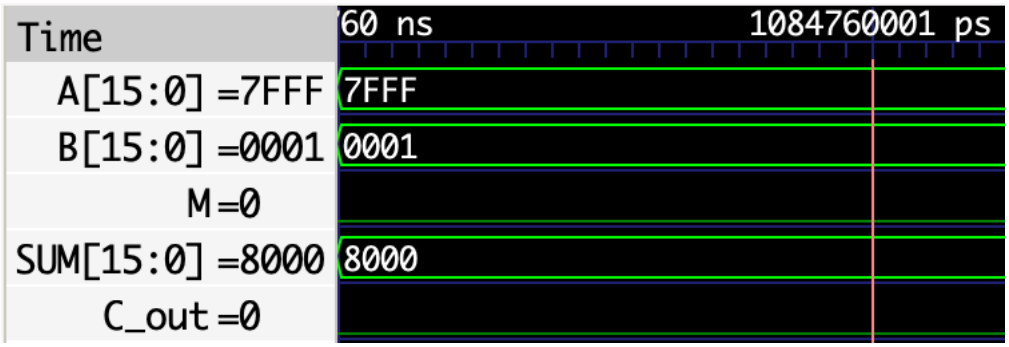
(ADD carry out): A=ffff B=0001 M=0 SUM=0000 C_out=1



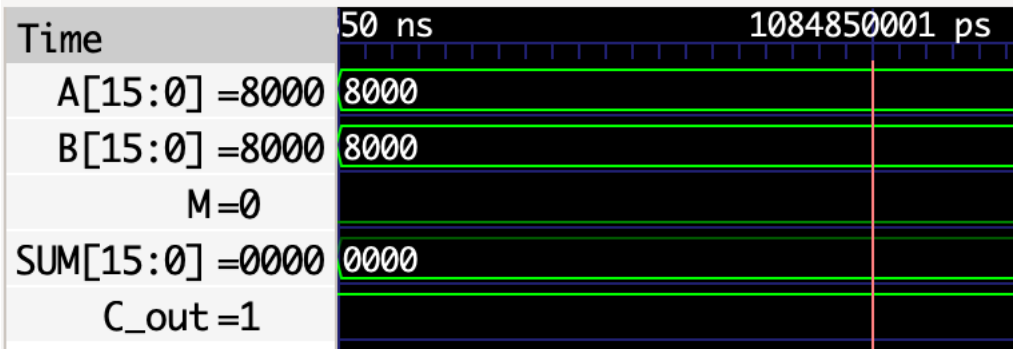
(ADD max plus max): A=ffff B=ffff M=0 SUM=fffe C_out=1



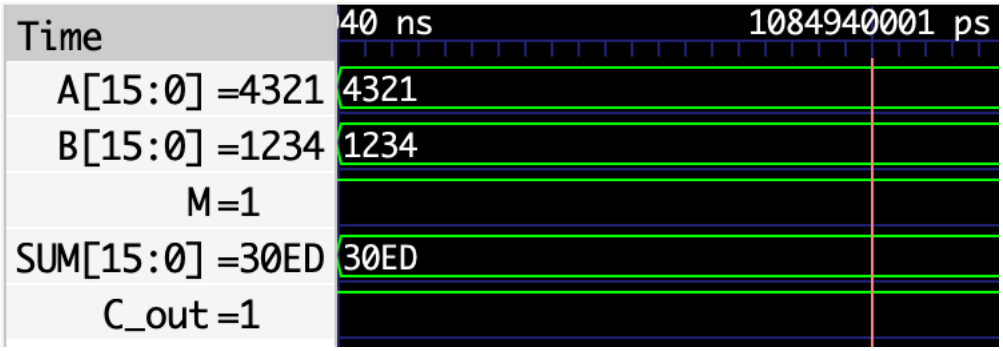
(ADD positive signed overflow): A=7fff B=0001 M=0 SUM=8000 C_out=0



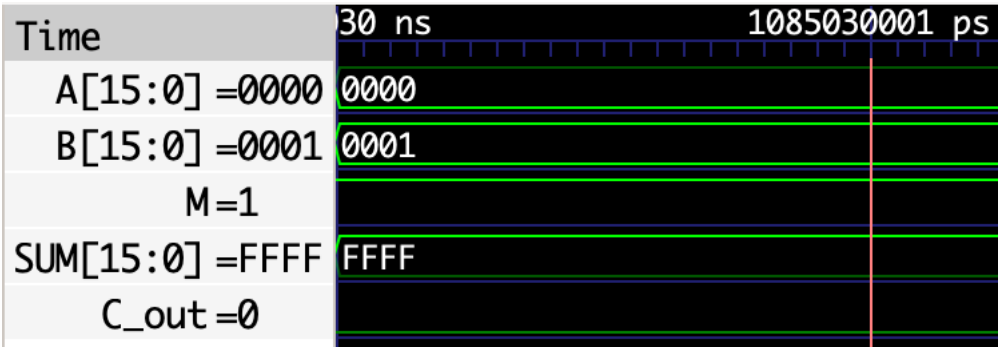
(ADD negative signed overflow): A=8000 B=8000 M=0 SUM=0000 C_out=1



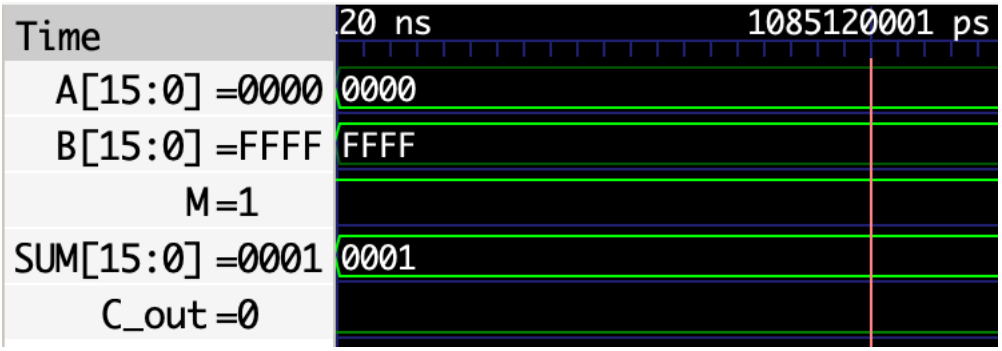
(SUB basic): A=4321 B=1234 M=1 SUM=30ed C_out=1



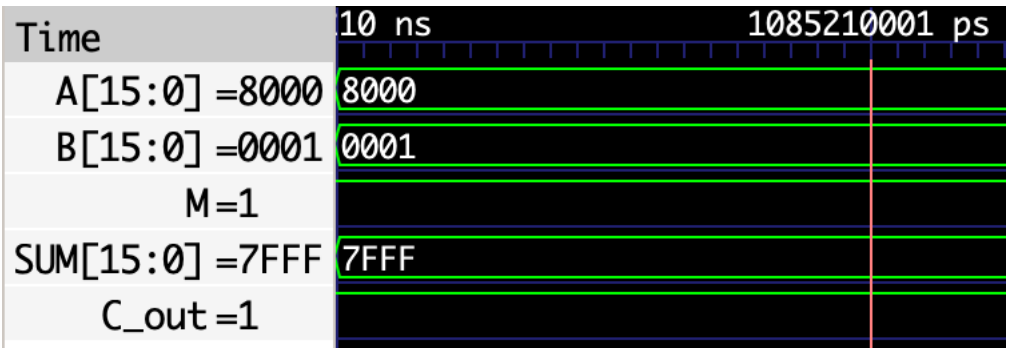
(SUB borrow): A=0000 B=0001 M=1 SUM=ffff C_out=0



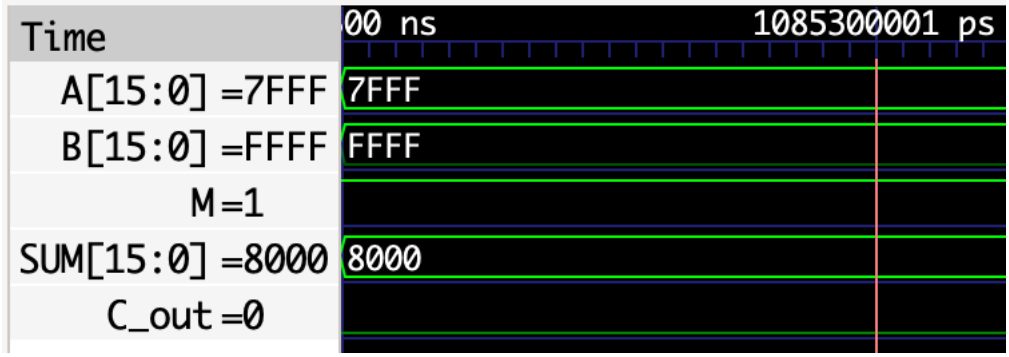
(SUB unsigned underflow edge): A=0000 B=ffff M=1 SUM=0001 C_out=0



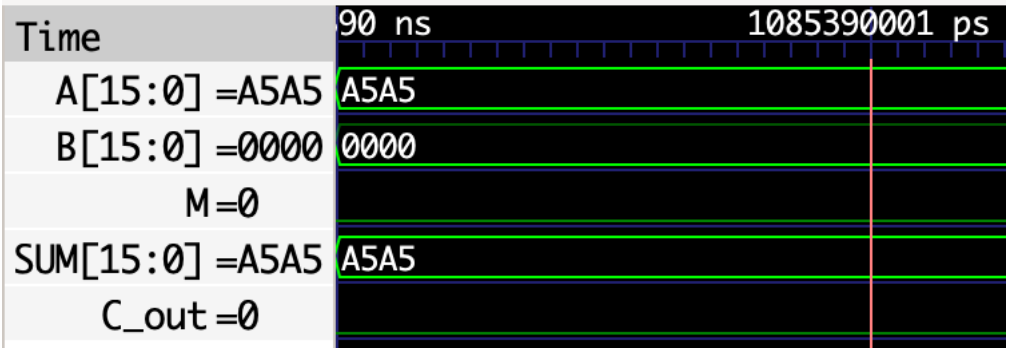
(SUB negative signed overflow): A=8000 B=0001 M=1 SUM=7fff C_out=1



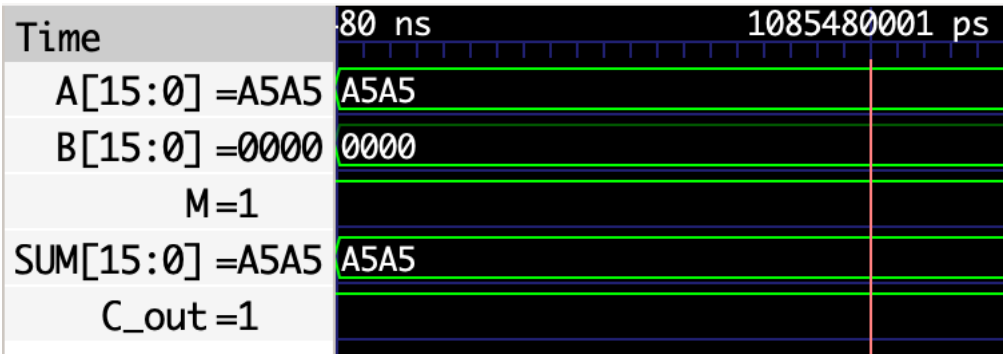
(SUB positive signed overflow): A=7fff B=ffff M=1 SUM=8000 C_out=0



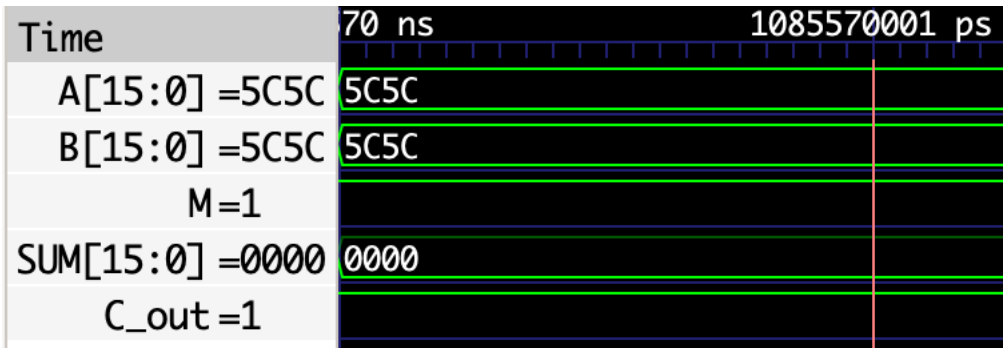
(ADD identity): A=a5a5 B=0000 M=0 SUM=a5a5 C_out=0



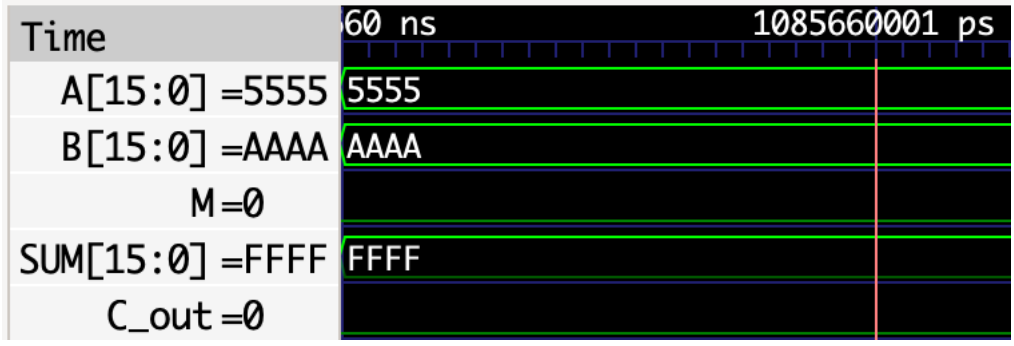
(SUB identity): A=a5a5 B=0000 M=1 SUM=a5a5 C_out=1



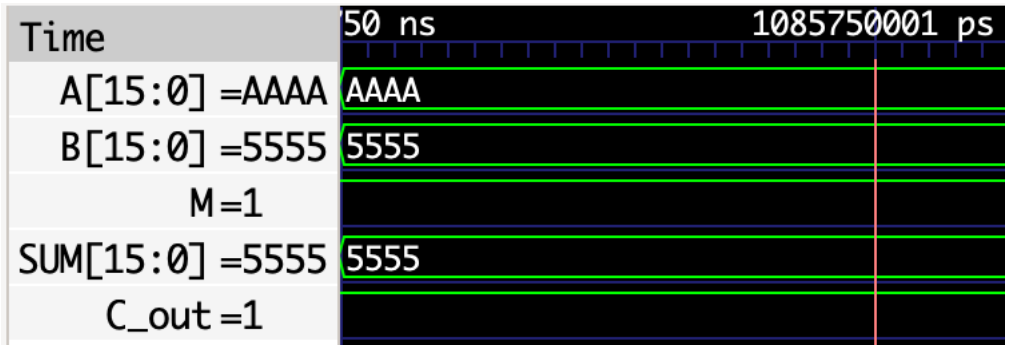
(SUB self): A=5c5c B=5c5c M=1 SUM=0000 C_out=1



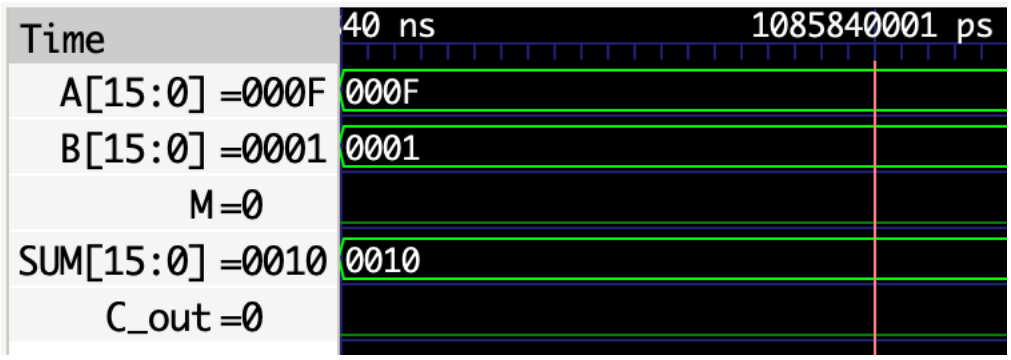
(ADD alternating bits): A=5555 B=aaaa M=0 SUM=ffff C_out=0



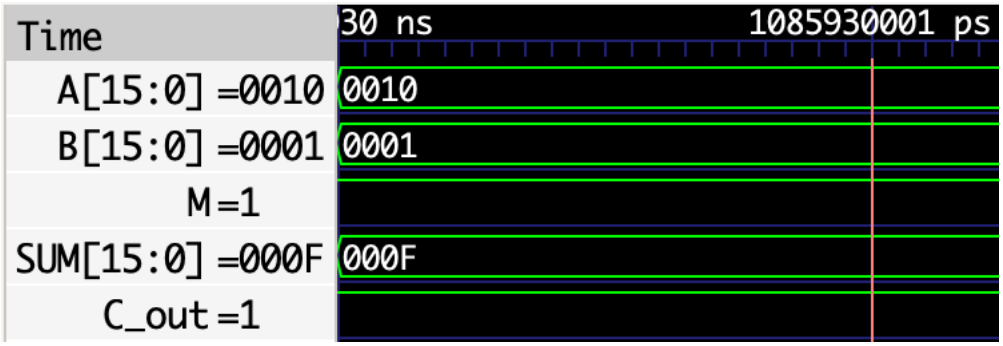
(SUB alternating bits): A=aaaa B=5555 M=1 SUM=5555 C_out=1



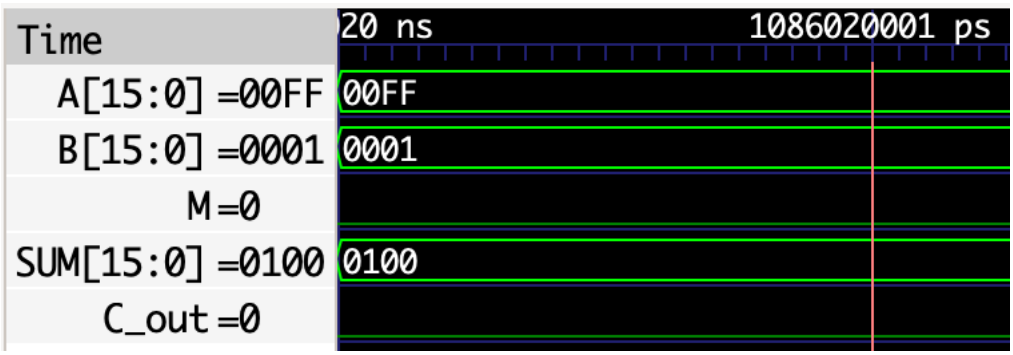
(ADD across block boundary): A=000f B=0001 M=0 SUM=0010 C_out=0



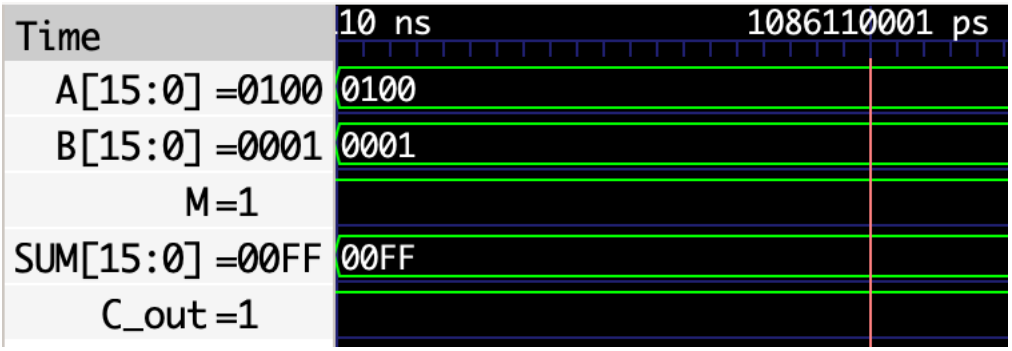
(SUB across block boundary): A=0010 B=0001 M=1 SUM=000f C_out=1



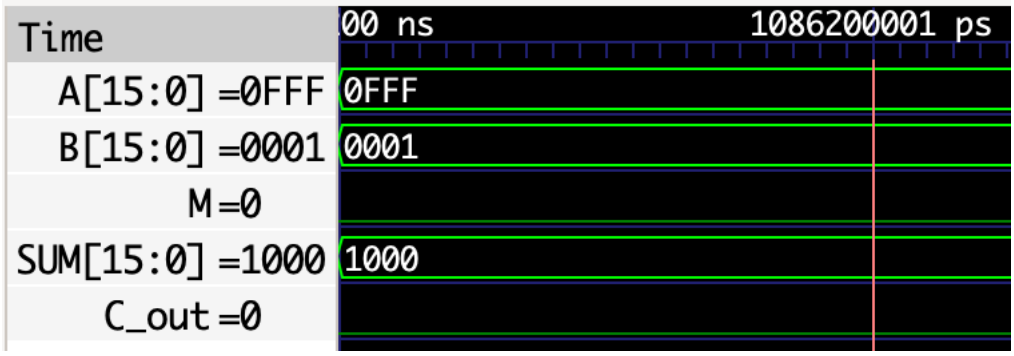
(ADD across block boundary): A=00ff B=0001 M=0 SUM=0100 C_out=0



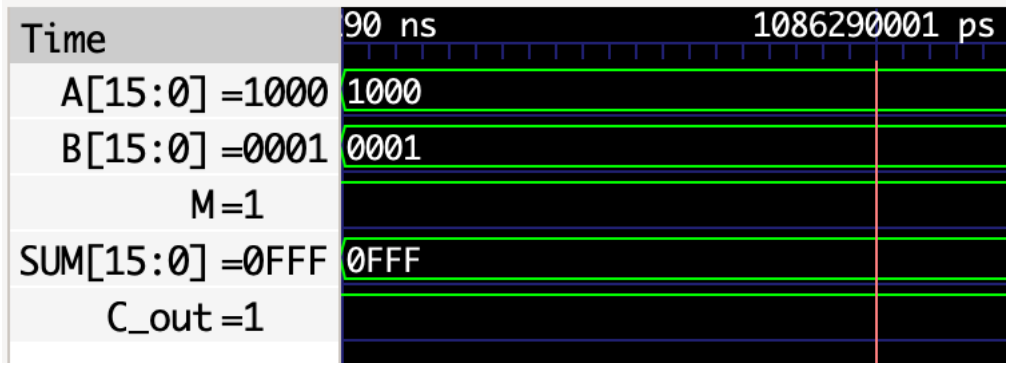
(SUB across block boundary): A=0100 B=0001 M=1 SUM=00ff C_out=1



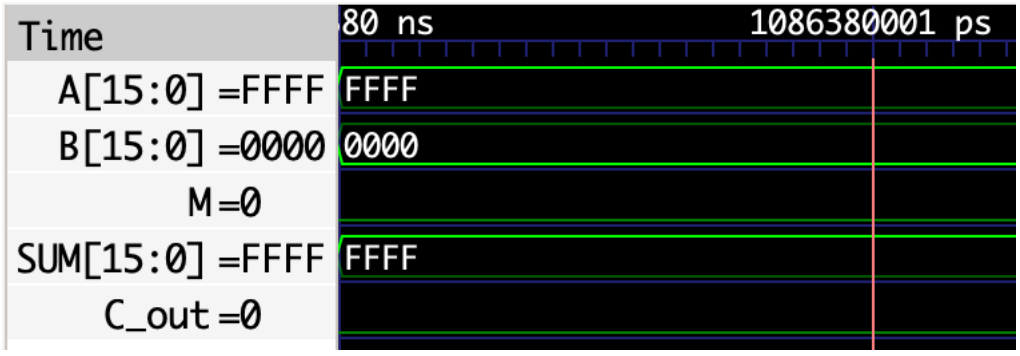
(ADD across block boundary): A=0fff B=0001 M=0 SUM=1000 C_out=0



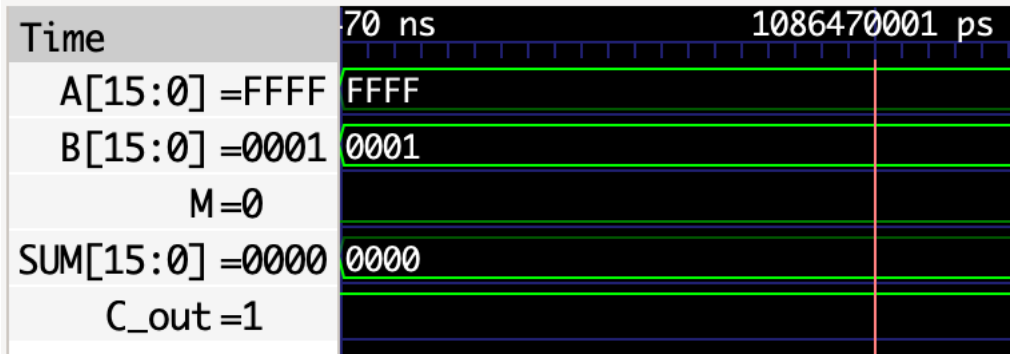
(SUB across block boundary): A=1000 B=0001 M=1 SUM=0fff C_out=1



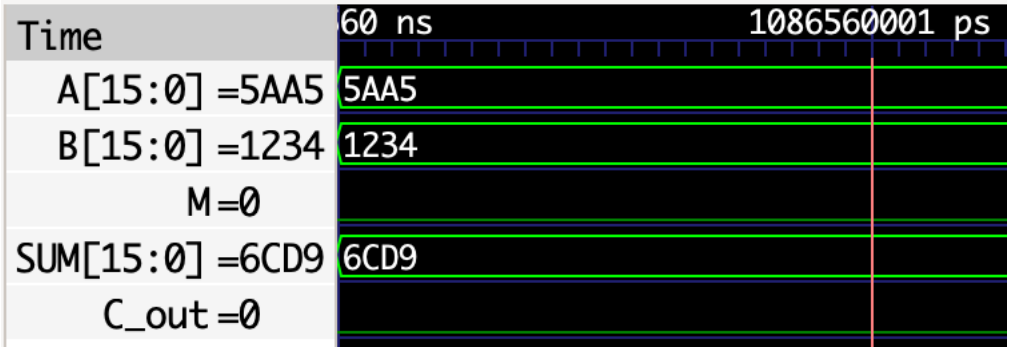
(Carry propagation baseline): A=ffff B=0000 M=0 SUM=ffff C_out=0



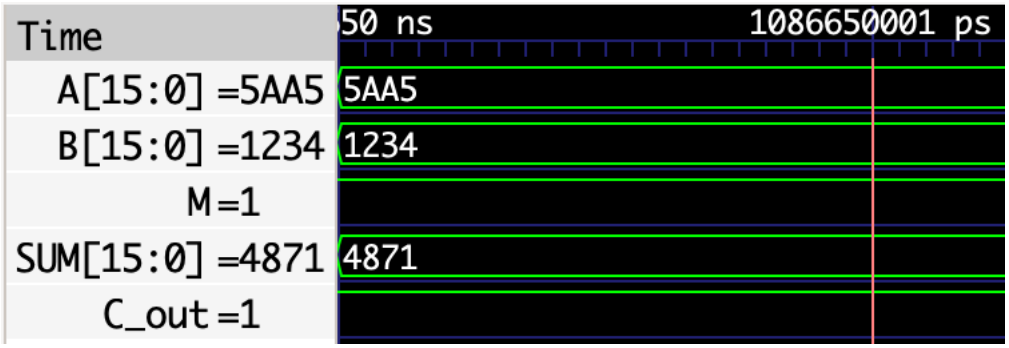
(Carry propagation transition): A=ffff B=0001 M=0 SUM=0000 C_out=1



(Mode transition add): A=5aa5 B=1234 M=0 SUM=6cd9 C_out=0

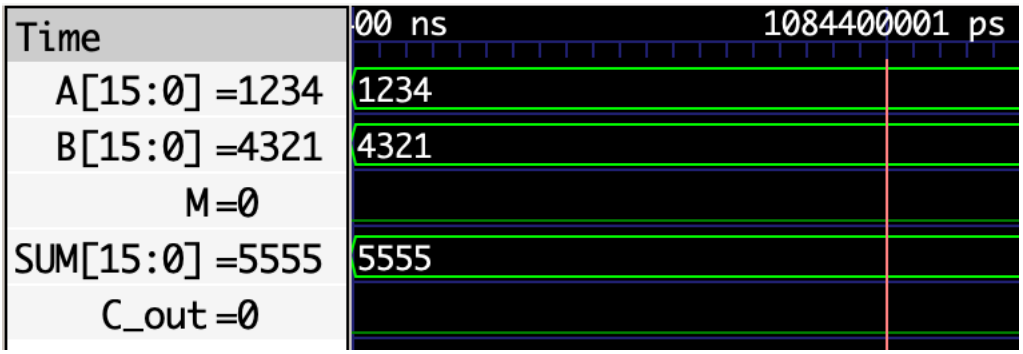


(Mode transition subtract): A=5aa5 B=1234 M=1 SUM=4871 C_out=1

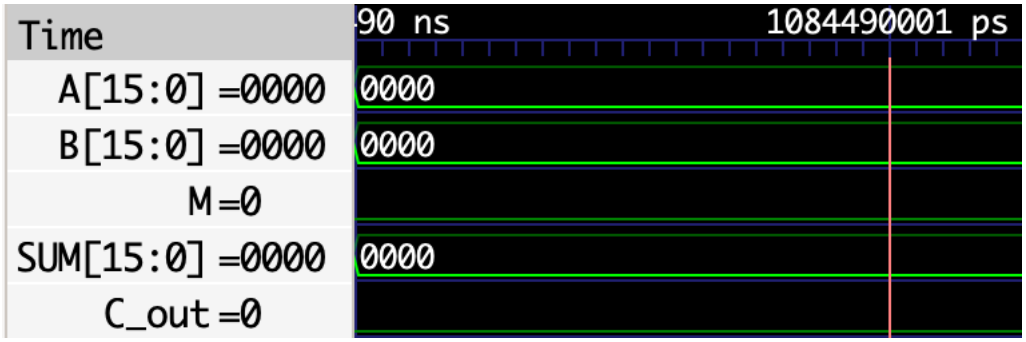


Signed with Overflow:

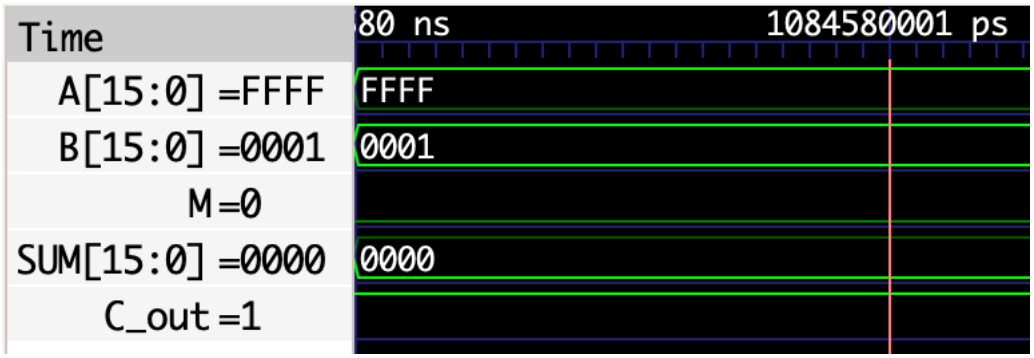
(ADD basic): A=1234 B=4321 M=0 SUM=5555 C_out=0 V=0



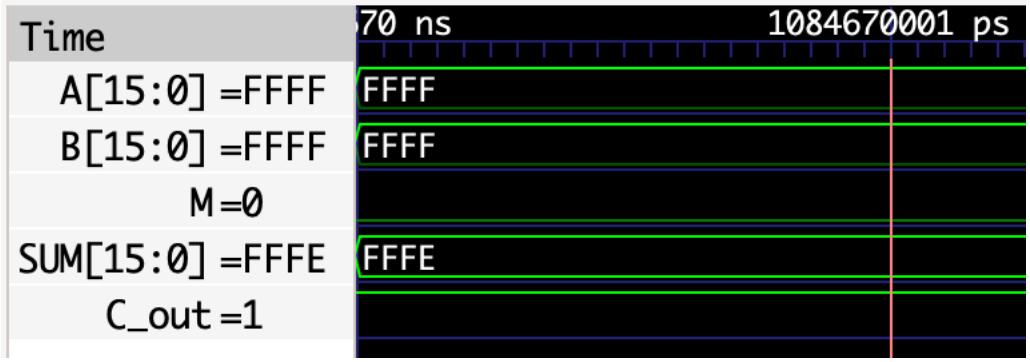
(ADD zero): A=0000 B=0000 M=0 SUM=0000 C_out=0 V=0



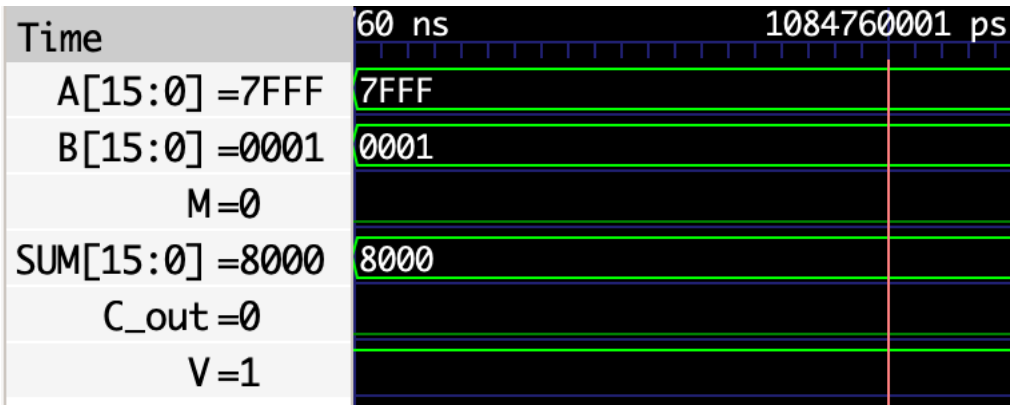
(ADD carry out): A=ffff B=0001 M=0 SUM=0000 C_out=1 V=0



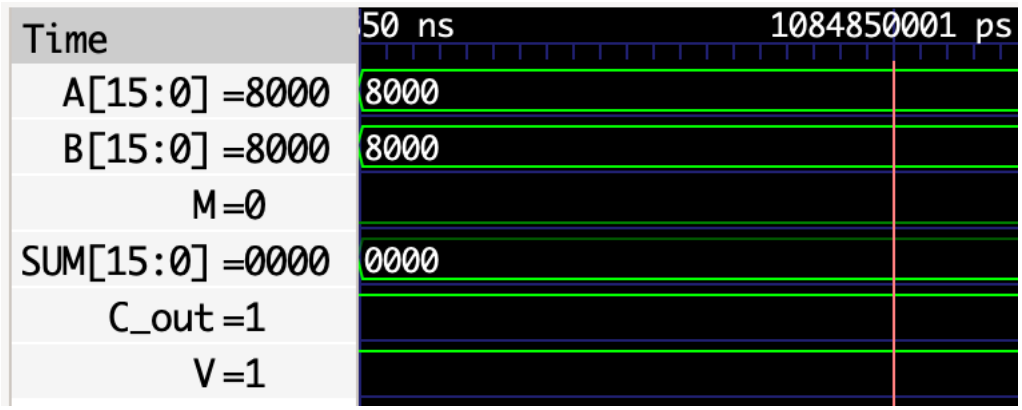
(ADD max plus max): A=ffff B=ffff M=0 SUM=fffe C_out=1 V=0



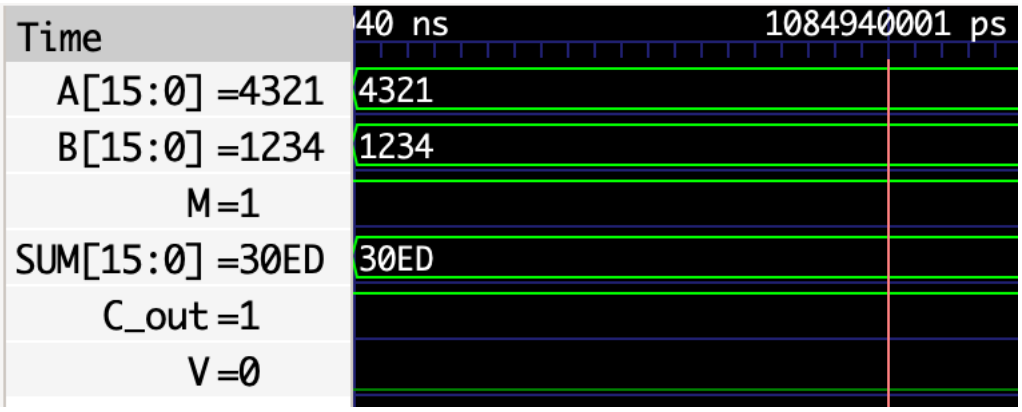
(ADD positive signed overflow): A=7fff B=0001 M=0 SUM=8000 C_out=0 V=1



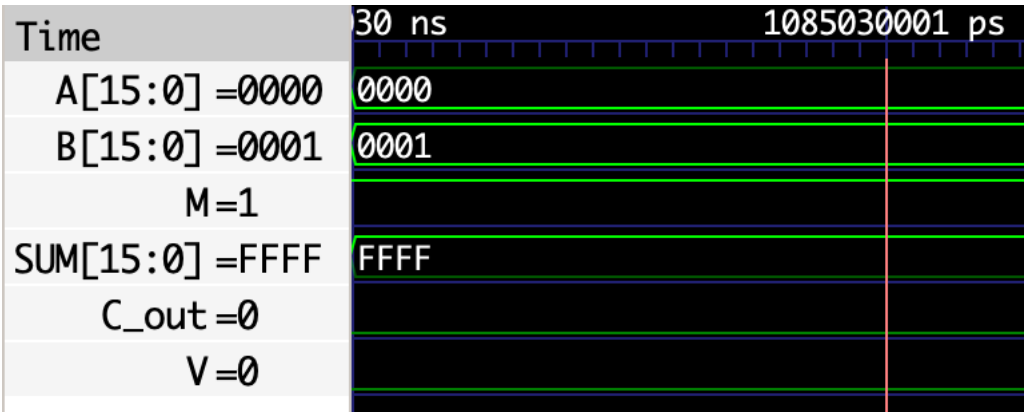
(ADD negative signed overflow): A=8000 B=8000 M=0 SUM=0000 C_out=1 V=1



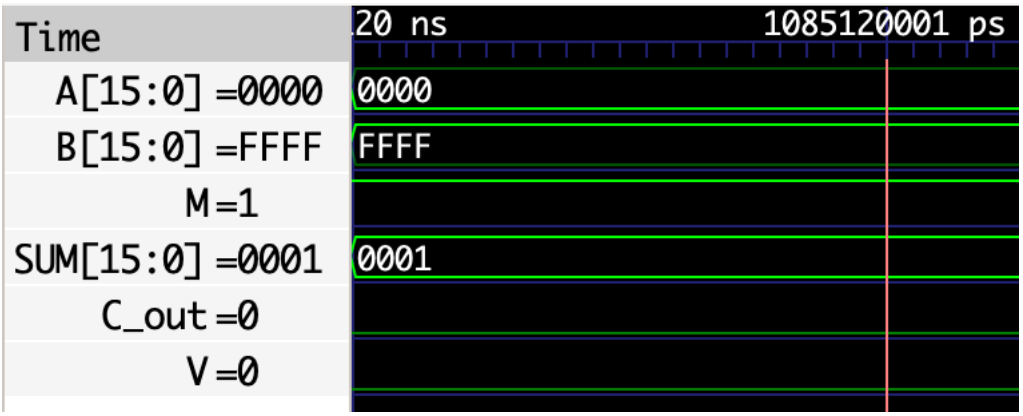
(SUB basic): A=4321 B=1234 M=1 SUM=30ed C_out=1 V=0



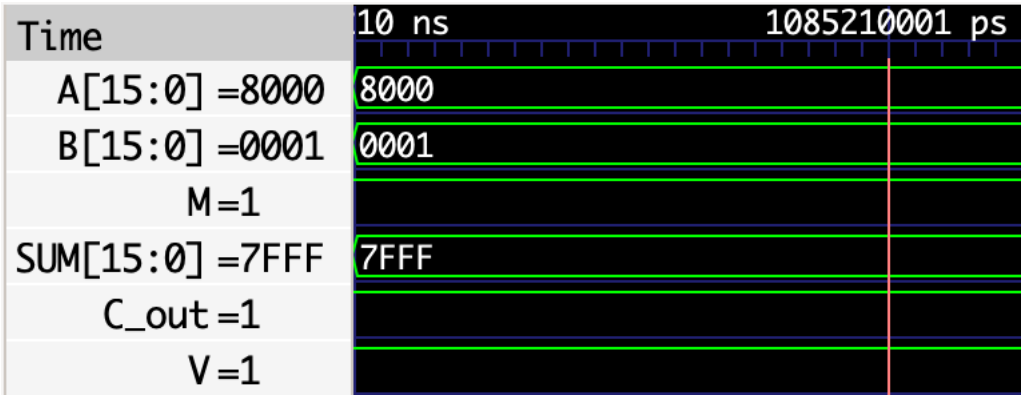
(SUB borrow): A=0000 B=0001 M=1 SUM=ffff C_out=0 V=0



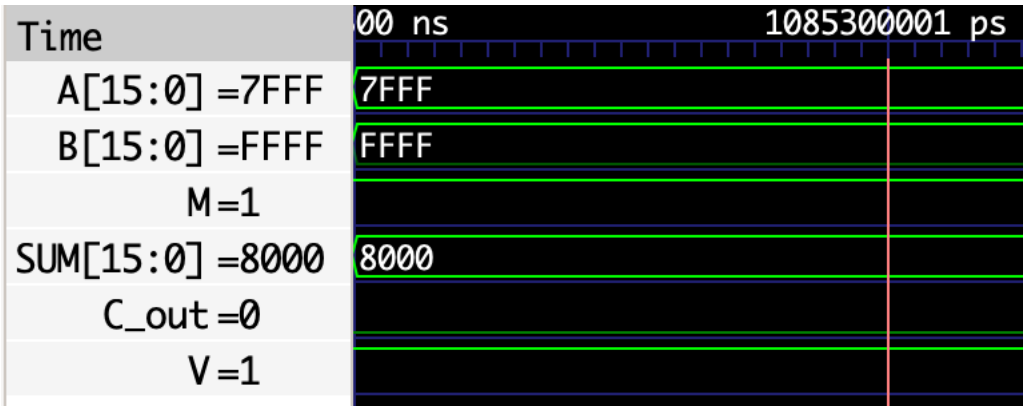
(SUB unsigned underflow edge): A=0000 B=ffff M=1 SUM=0001 C_out=0 V=0



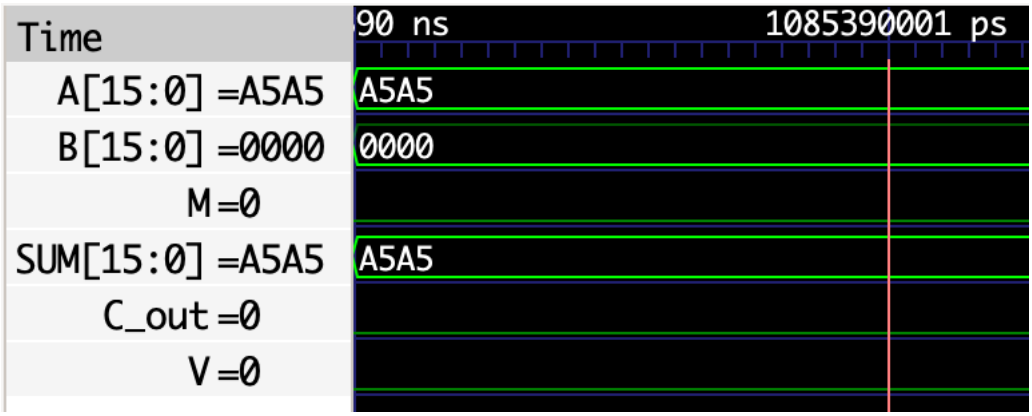
(SUB negative signed overflow): A=8000 B=0001 M=1 SUM=7fff C_out=1 V=1



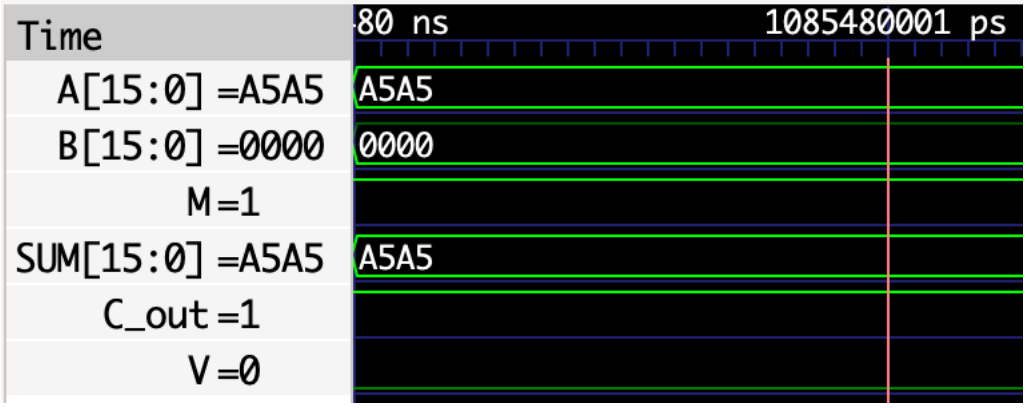
(SUB positive signed overflow): A=7fff B=ffff M=1 SUM=8000 C_out=0 V=1



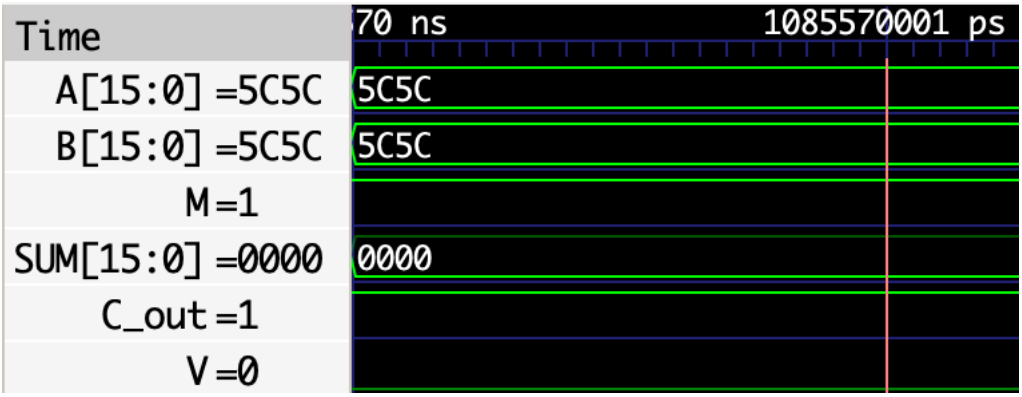
(ADD identity): A=a5a5 B=0000 M=0 SUM=a5a5 C_out=0 V=0



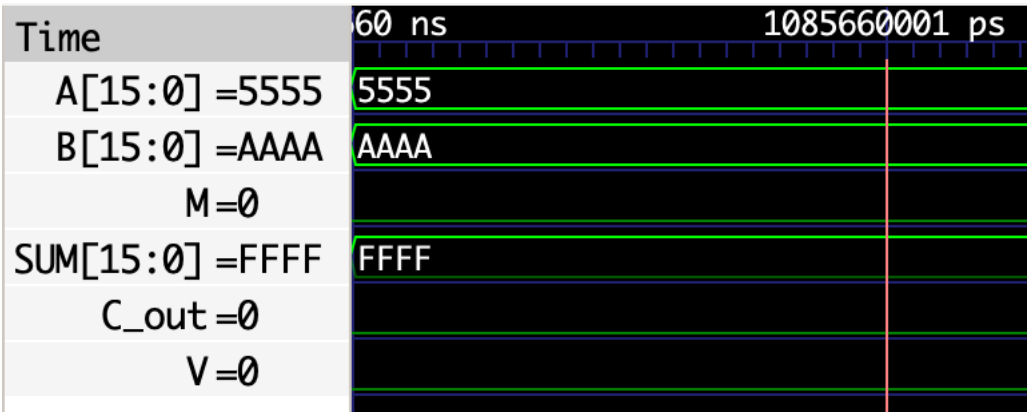
(SUB identity): A=a5a5 B=0000 M=1 SUM=a5a5 C_out=1 V=0



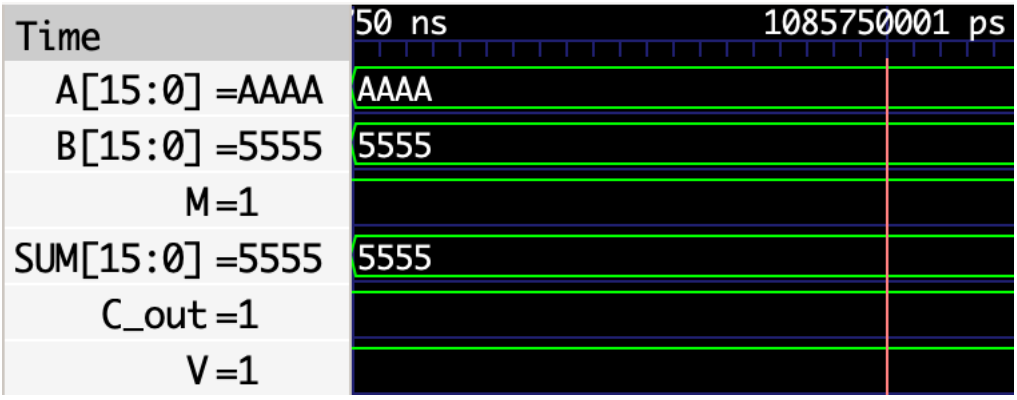
(SUB self): A=5c5c B=5c5c M=1 SUM=0000 C_out=1 V=0



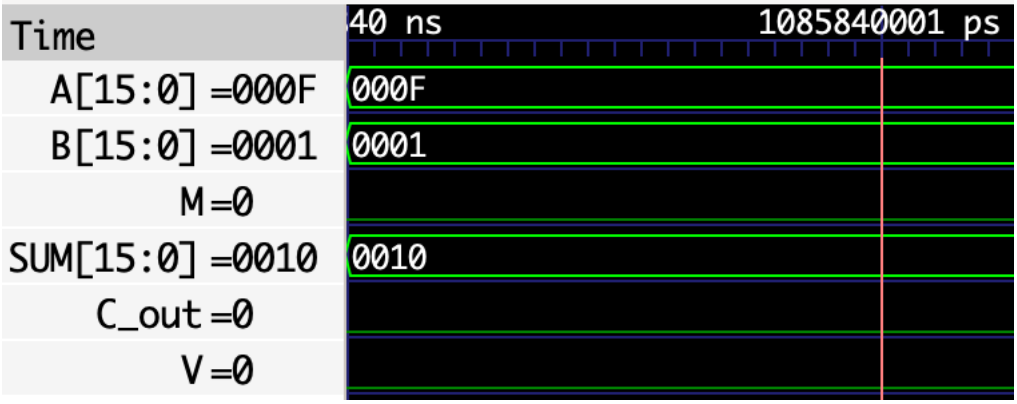
(ADD alternating bits): A=5555 B=aaaa M=0 SUM=ffff C_out=0 V=0



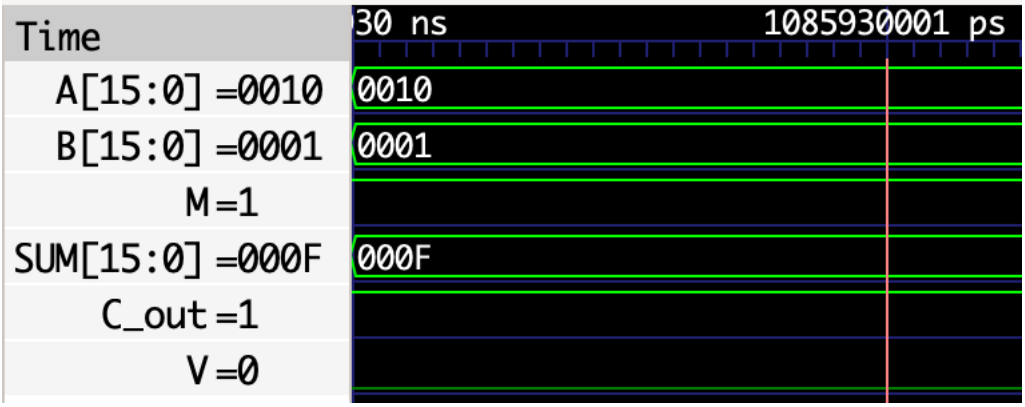
(SUB alternating bits): A=aaaa B=5555 M=1 SUM=5555 C_out=1 V=1



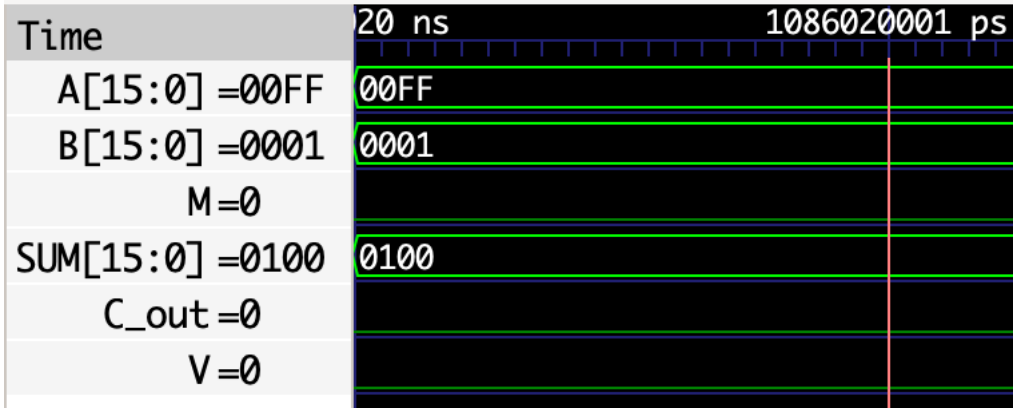
(ADD across block boundary): A=000f B=0001 M=0 SUM=0010 C_out=0 V=0



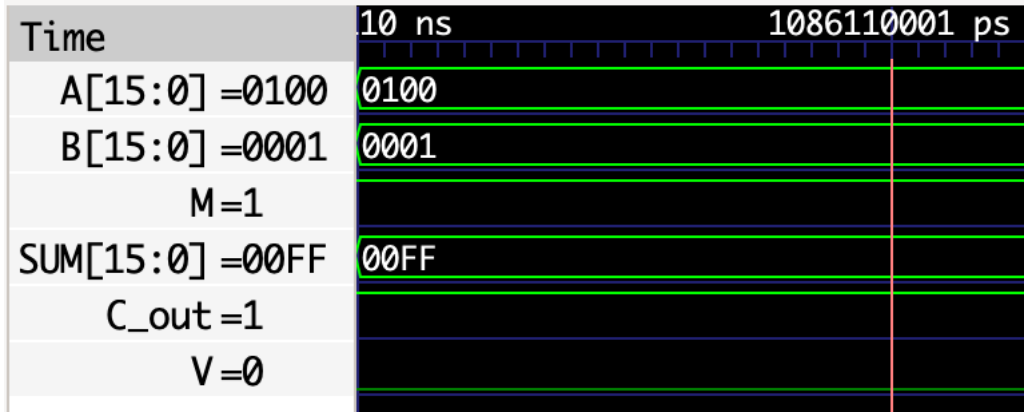
(SUB across block boundary): A=0010 B=0001 M=1 SUM=000f C_out=1 V=0



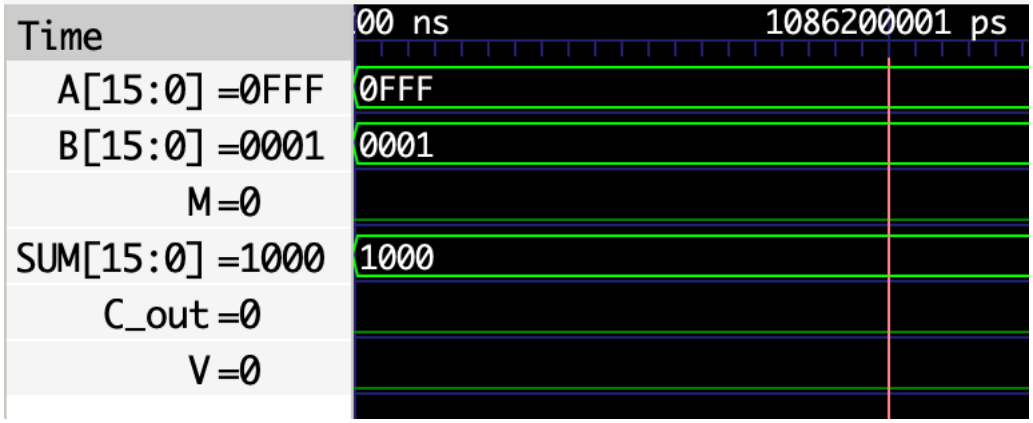
(ADD across block boundary): A=00ff B=0001 M=0 SUM=0100 C_out=0 V=0



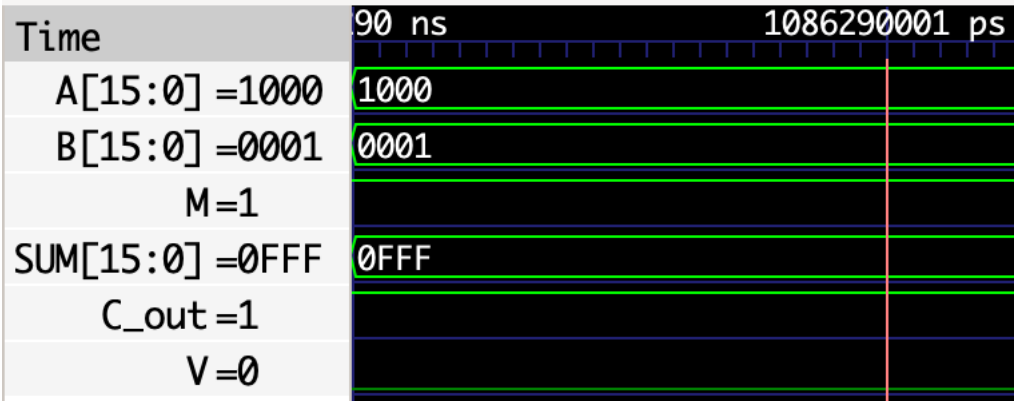
(SUB across block boundary): A=0100 B=0001 M=1 SUM=00ff C_out=1 V=0



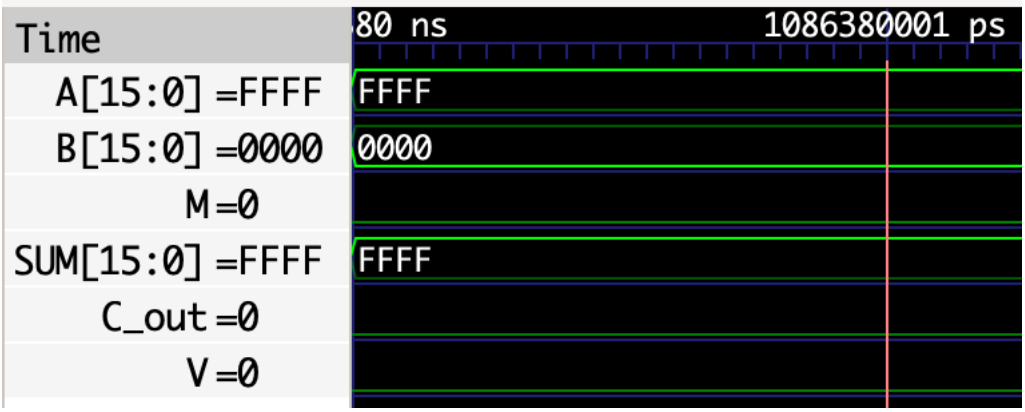
(ADD across block boundary): A=0fff B=0001 M=0 SUM=1000 C_out=0 V=0



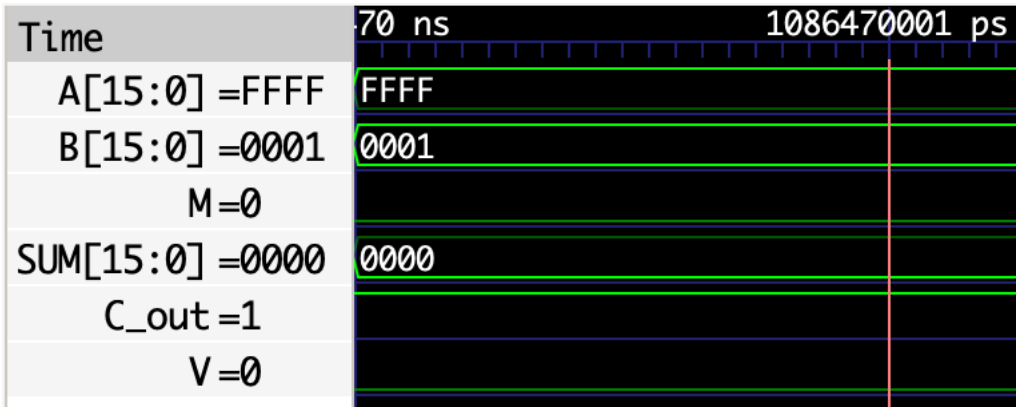
(SUB across block boundary): A=1000 B=0001 M=1 SUM=0fff C_out=1 V=0



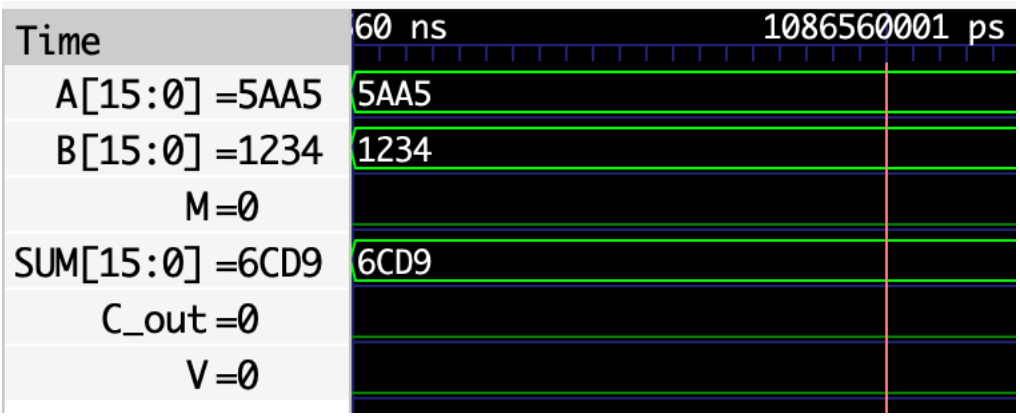
(Carry propagation baseline): A=ffff B=0000 M=0 SUM=ffff C_out=0 V=0



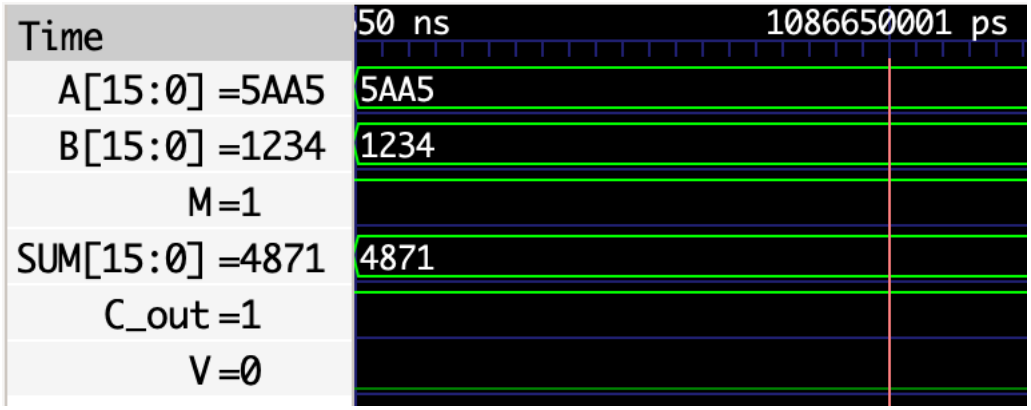
(Carry propagation transition): A=ffff B=0001 M=0 SUM=0000 C_out=1 V=0



(Mode transition add): A=5aa5 B=1234 M=0 SUM=6cd9 C_out=0 V=0



(Mode transition subtract): A=5aa5 B=1234 M=1 SUM=4871 C_out=1 V=0



Sum Stabilization and Propagation Delay

The waveforms show that the sum does not change immediately to its final value when the inputs become A=1234 and B=4321, with M=0. Instead, it temporarily retains the previous result and then passes through several intermediate values before settling at the correct sum, 5555. These temporary values occur because different signal paths have different propagation delays. Each sum bit depends on both its operand inputs and its incoming carry, which may arrive at different times.

In the RCA waveform, the carry propagates sequentially through the full-adder stages, causing the sum bits to settle at different times. In the CLA waveform, lookahead logic calculates carries more quickly, allowing the sum to stabilize sooner for this transition. Both circuits can exhibit temporary glitches; these are expected in a combinational circuit with gate delays and do not indicate an incorrect final result. The testbench therefore waits for the settling interval before checking the outputs.



VI. Experience and Reflection

Since I already have experience designing adders, I set a few rules for myself, I am only allowed to use AND gates with maximum of 6 inputs and had to build easily scalable building blocks.

Comparing RCA and CLA taught me about the tradeoff between hardware complexity and propagation delay. The RCA was easier to construct and trace because each stage passes its carry to the next. However, this dependency creates a long carry path. The CLA required more logic and more careful wiring, especially when generating PG and GG signals and connecting the second-level carry-lookahead generator. I initially thought that four 4-bit CLA blocks meant there were only four carry-lookahead generators, but the complete design uses a fifth generator to calculate carries between blocks. Understanding this hierarchy helped me trace the critical path more accurately.

Another improvement came from recognizing that the individual full adders' carry-out calculations were unnecessary in the CLA. Since the carry-lookahead generators already produced the required carries, I replaced the full adders with no_Cout_adder cells that retained only the sum, propagate, and generate logic. This reduced the gate count without changing the arithmetic behavior. It showed me that improving a design

involves identifying redundant logic as well as choosing a faster architecture. Upon writing this report, I realized that I at least one possible improvement can be made. The final bit of each 4-bit block in the CLA implementation is calculated twice, I could save up to 20 logic gates by removing this redundant circuit. I will sure apply this to my design if I have the chance to design a high bit count circuit.

Verification, which as always, the most tedious part, also required changes during the design process. I initially wanted to test every possible input combination, but two 16-bit operands and two operation modes require more than 8.5 billion cases per design. I therefore used repeatable random tests, block-boundary sweeps, and selected corner cases. Automated comparisons made it easier to detect errors, while aligned terminal output and waveform files helped me inspect individual operations. I also learned to distinguish unsigned carry from signed overflow: the same binary result can be valid under one interpretation and invalid under the other.

Timing analysis reinforced the importance of using consistent assumptions. After clarifying the gate delays, I used 5 ns for XOR gates and 2 ns for AND and OR gates and allowed sufficient settling time in the testbenches. Tracing the paths also helped me understand why adding overflow detection does not necessarily increase the overall critical-path delay: in the CLA, the overflow XOR operates in parallel with the final sum XOR. Overall, this project developed my understanding of modular gate-level design, verification, and the relationship between circuit structure, gate count, and timing.

AI Usage

Use 1: used to add delay to code
Prompt: “add delay for each logic gate, 1 for inverter, 2 for and, 2 for or, 5 for xor”

Use 2: used to generate some pictures for report

Use 3: used to calculate delay
Prompt: “calculate the longest delay of each circuit and return a graph”

Use 4: used to give example of longest path
Prompt: “give a way to use the longest path of each circuit”

Use 5: used to give possible corner cases
Prompt: “generate a list of possible corner cases for signed and unsigned implementation”